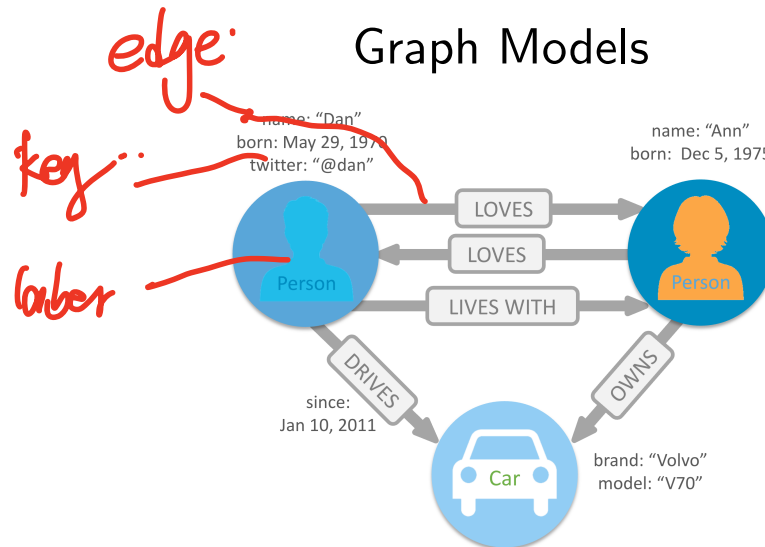


COMPSCI 752

Big Data Management

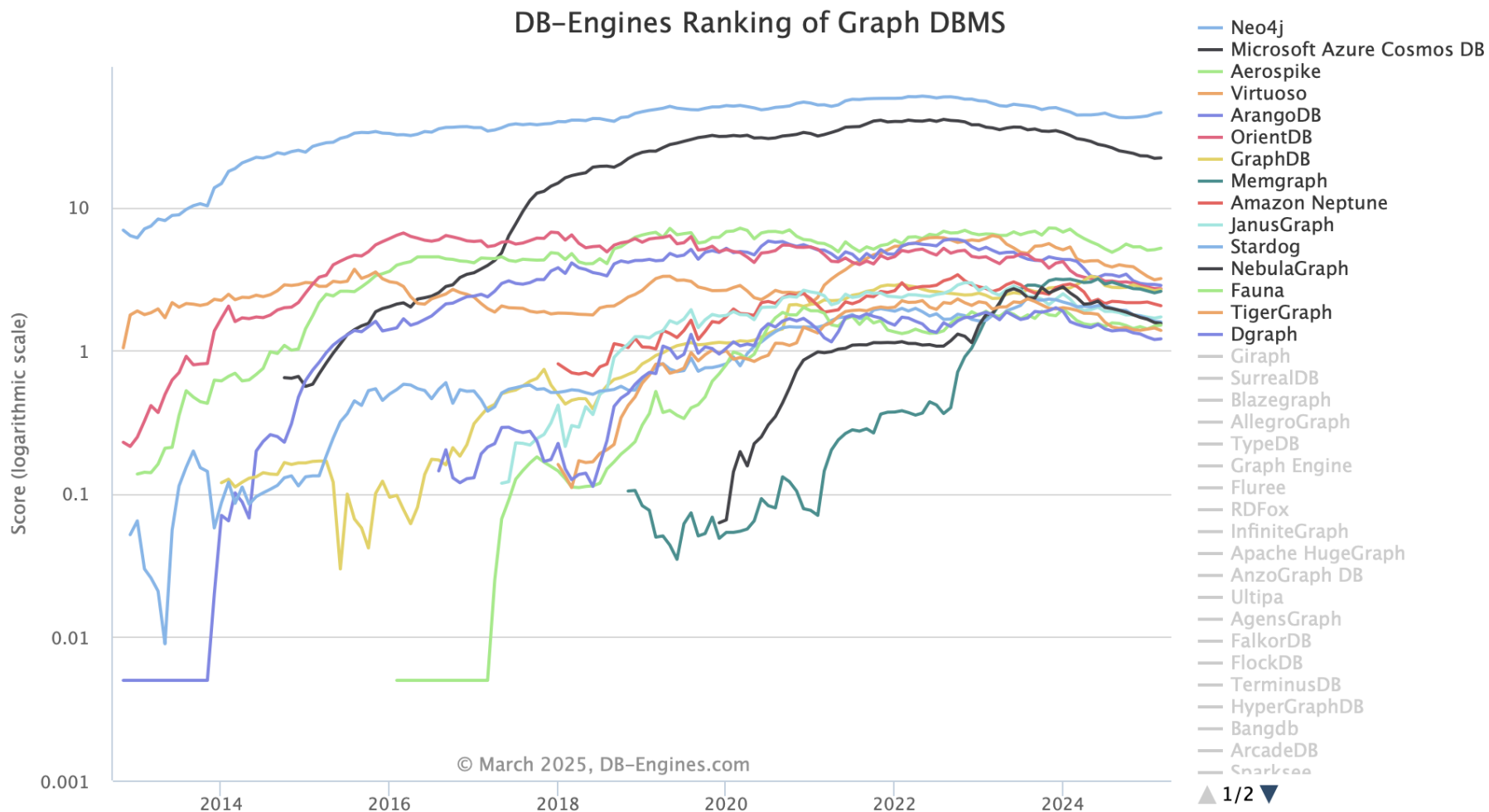
Instructor: s.link@auckland.ac.nz



expressive.

不存在不一定
明白。

Popularity trends for Graph database engines



Property Graph Model

- Implemented in many systems
- Used as a reference model for various research work.

Property Graph Model

- Implemented in many systems
- Used as a reference model for various research work.

Aim

- Introduce the basic concepts of the property graph model, following the LDBC's Graph Query Language Task Force
- Discuss a number of variations of the property graph in terms of specializations and structural extensions
- Illustrate how the property graph model fits into the family of existing graph models

Property Graph Model

Data is represented as a directed, attributed multi-graph

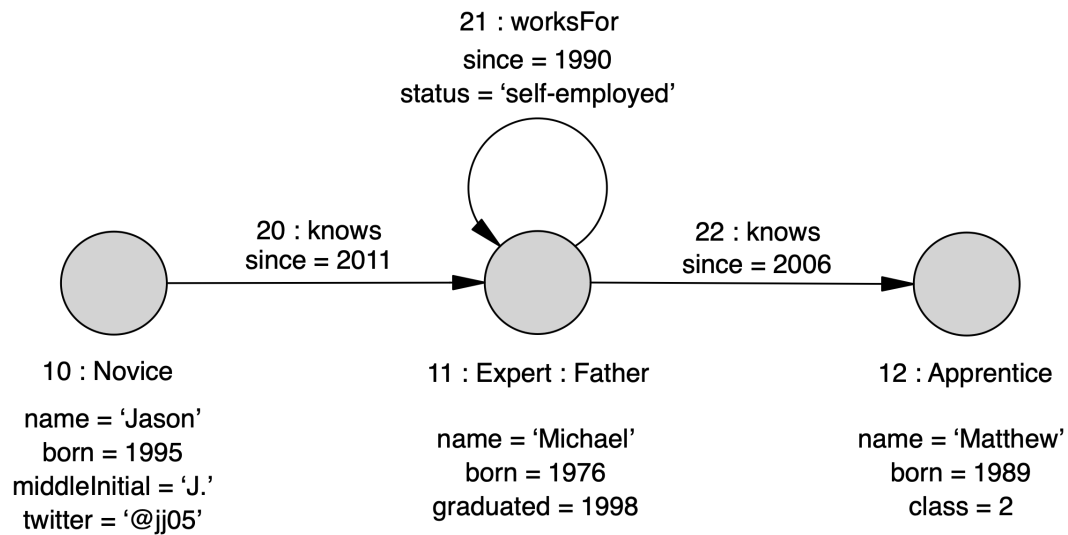
Property Graph Model

Data is represented as a directed, attributed multi-graph

Vertices and edges are rich objects with:

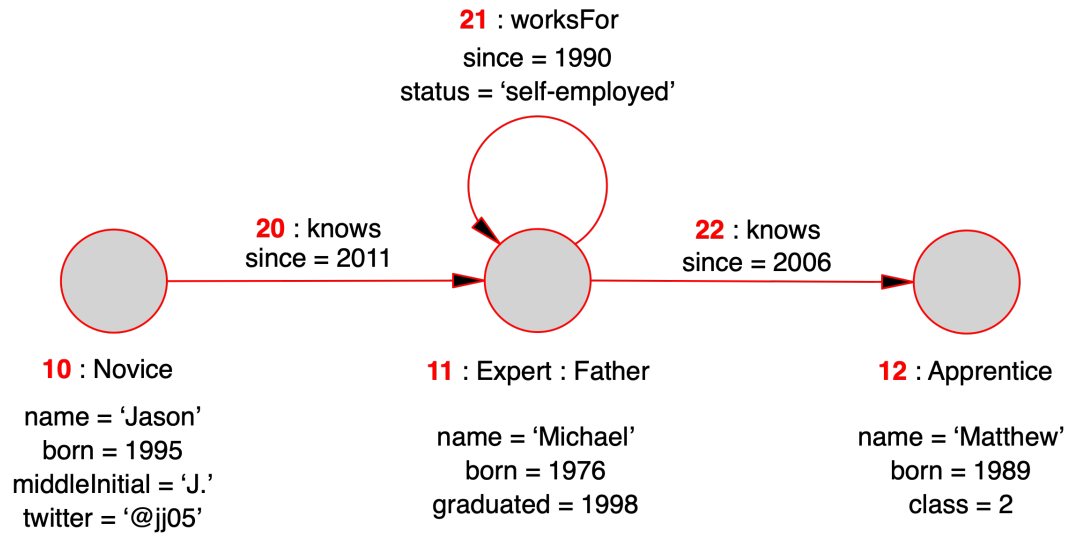
- a set of labels and
- a set of key-value pairs, called *properties*

Example



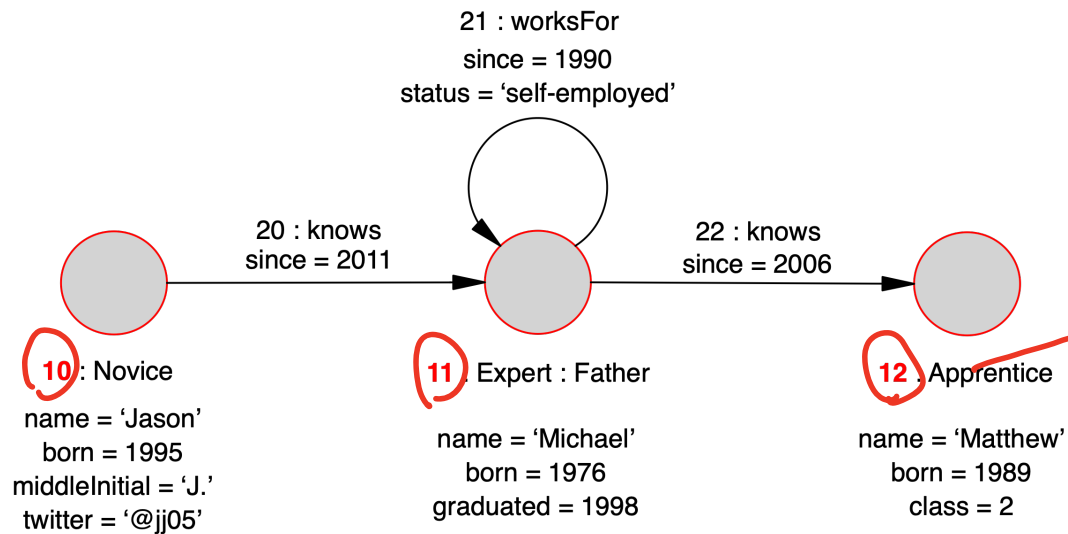
Property graph

Example



Objects

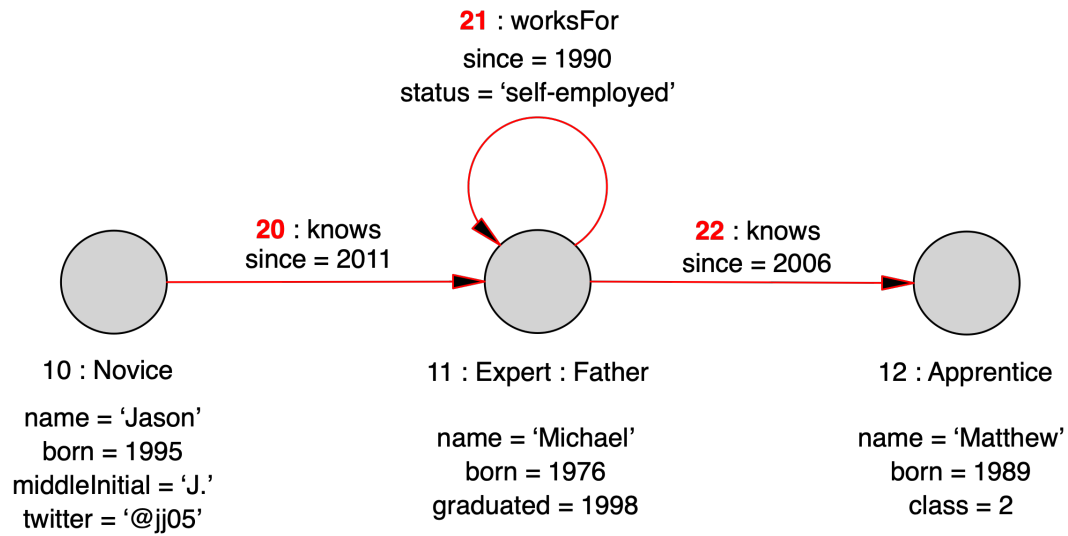
Example



Vertices

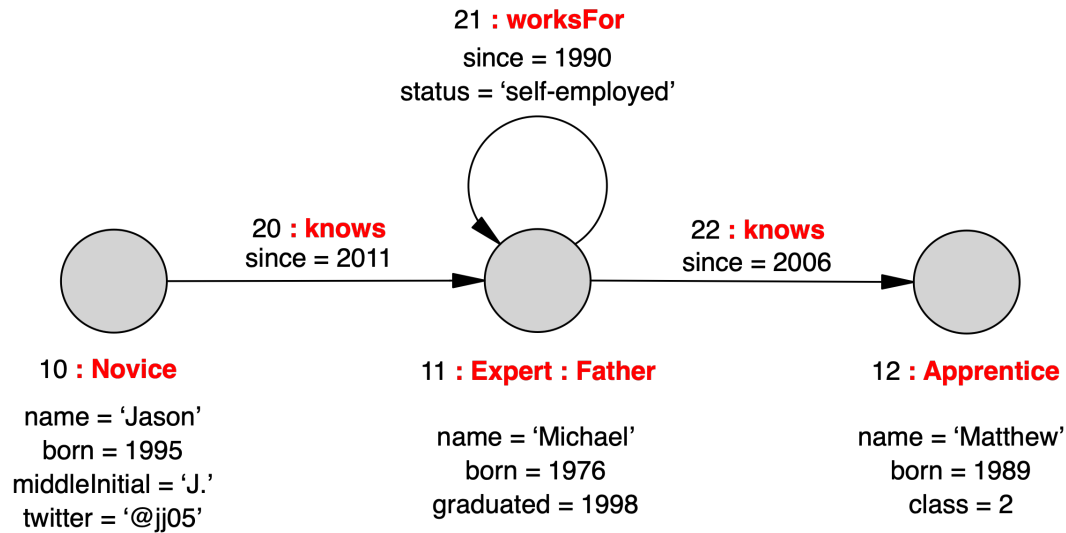
顶点

Example



Edges

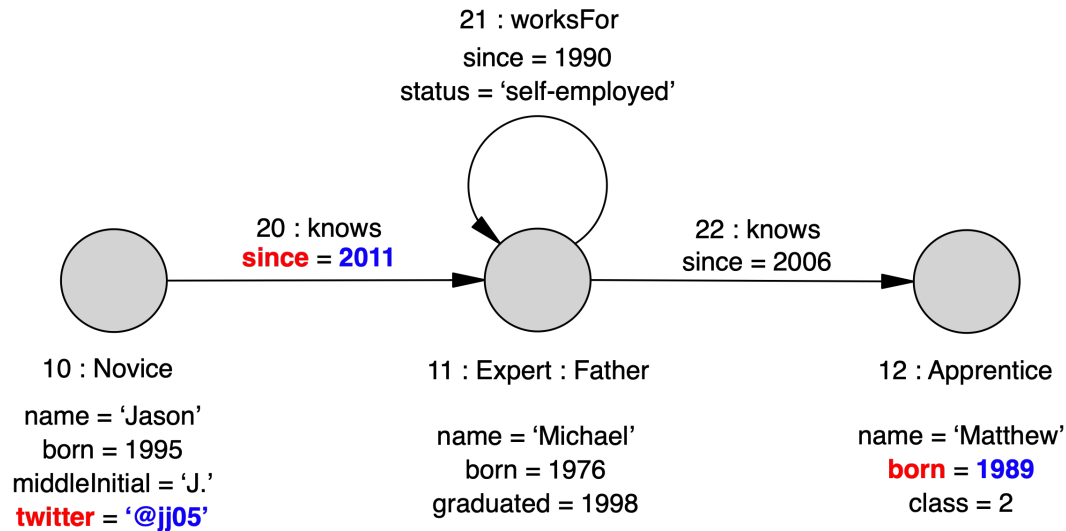
Example



Labels

- Descriptive information to which class objects belong
- Indicate which type of real-world entity objects describe

Example



Key-Value Pairs

- **Key** specify the meaning of the property
- **Value** specify the value of the property

Property Graph Model - Formal Definition

Notation

We assume the following sets are pairwise disjoint.

- $\mathcal{O}$ is a set of objects,
- $\mathcal{L}$ be a finite set of labels,
- $\mathcal{K}$ is a set of property keys, and
- $\mathcal{N}$ is a set of values.

Property Graph Model - Formal Definition

Notation

We assume the following sets are pairwise disjoint.

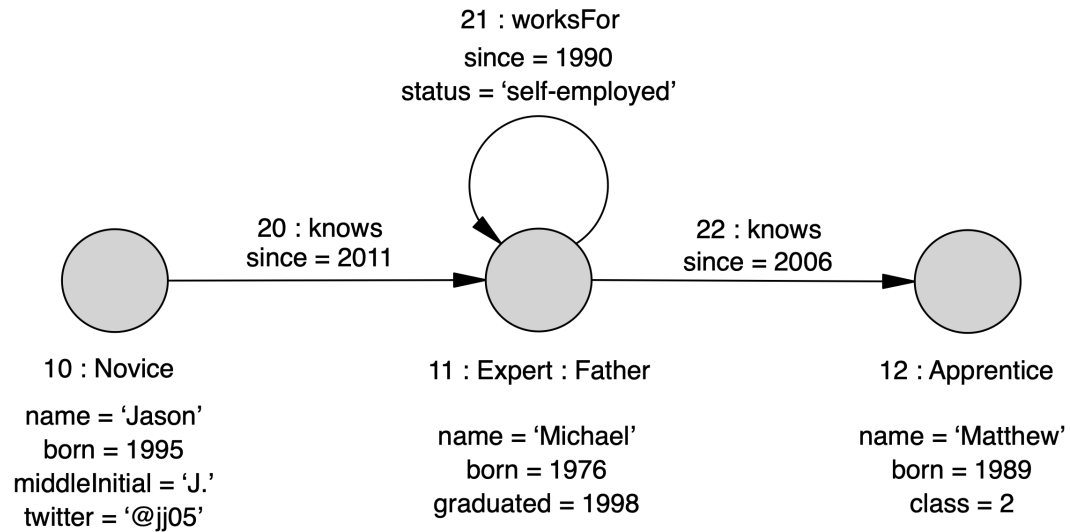
- $\mathcal{O}$ is a set of objects,
- $\mathcal{L}$ be a finite set of labels,
- $\mathcal{K}$ is a set of property keys, and
- $\mathcal{N}$ is a set of values.

Definition (Property graph)

A structure $(V, E, \eta, \lambda, \nu)$ where

- $V \subseteq \mathcal{O}$ is a finite set of objects, called *vertices*,
- $E \subseteq \mathcal{O}$ is a finite set of objects, called *edges*,
- $\eta : E \rightarrow V \times V$ is a function assigning to each edge an ordered pair of vertices,
- $\lambda : V \cup E \rightarrow \mathcal{P}(\mathcal{L})$ is a function assigning to each object a finite set of labels, and
- $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{N}$ is a partial function assigning values for properties to objects, such that the set of domain values where ν is defined is finite.

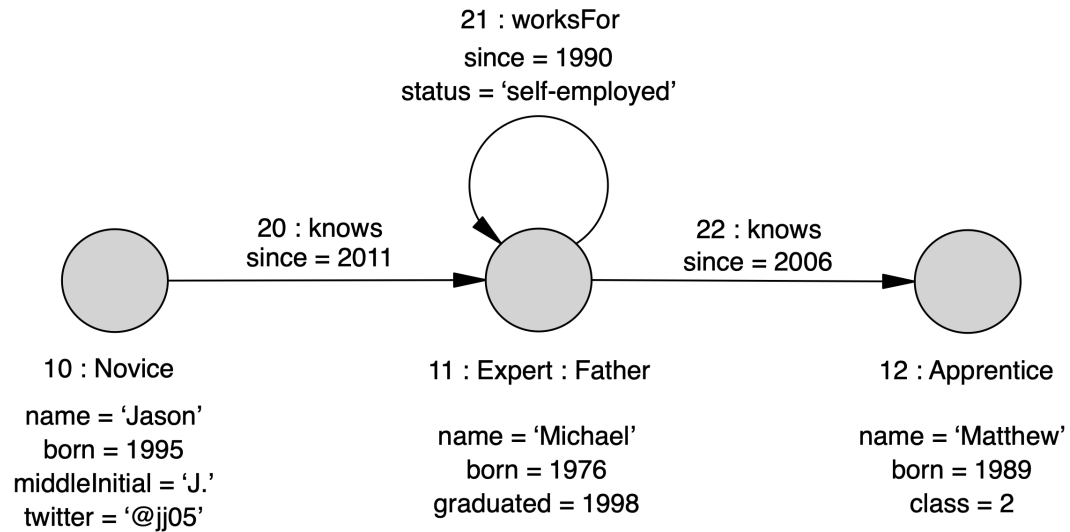
Example for Formal Definition



Vertex set V

10, 11, 12

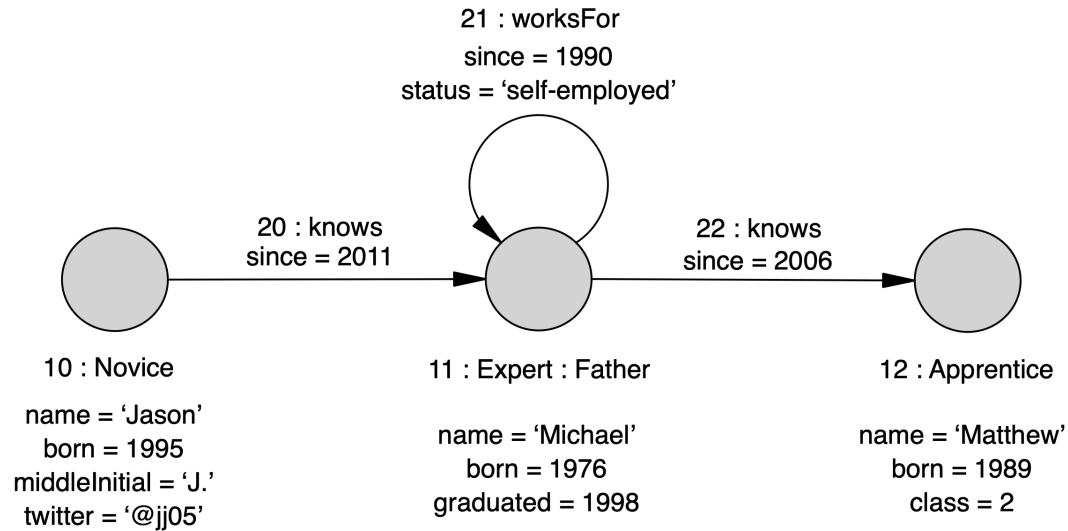
Example for Formal Definition



Edge set E

20, 21, 22

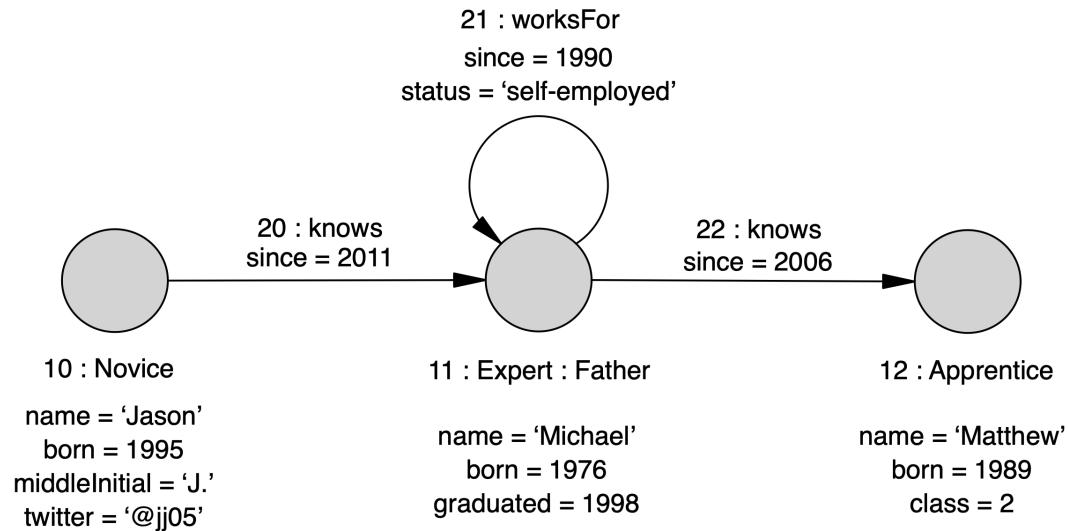
Example for Formal Definition



Edge relationship $\eta : E \rightarrow V \times V$

$20 \mapsto (10, 11), 21 \mapsto (11, 11),$
 $22 \mapsto (11, 12)$

Example for Formal Definition



Labelling $\lambda : V \cup E \rightarrow \mathcal{P}(\mathcal{L})$

$10 \mapsto \{\text{Novice}\},$

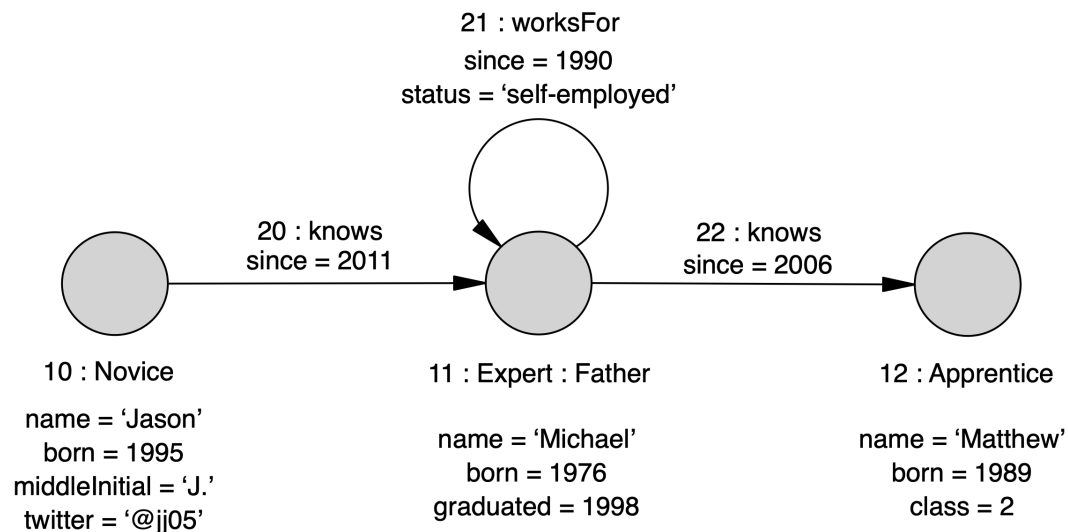
$11 \mapsto \{\text{Expert, Father}\},$

$12 \mapsto \{\text{Apprentice}\},$

$20 \mapsto \{\text{knows}\}, 21 \mapsto \{\text{worksFor}\},$

$22 \mapsto \{\text{knows}\}$

Example for Formal Definition



Value assignment

$$\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{N}$$

$(10, \text{name}) \mapsto \text{'Jason'},$
 $(10, \text{born}) \mapsto 1995,$
 $(10, \text{middleInitial}) \mapsto \text{'J.'},$
 $(10, \text{twitter}) \mapsto \text{'@jj05'},$
 $(11, \text{name}) \mapsto \text{'Michael'},$
 $(11, \text{born}) \mapsto 1976,$
 $(11, \text{graduated}) \mapsto 1998,$
 $(12, \text{name}) \mapsto \text{'Matthew'},$
 $(12, \text{born}) \mapsto 1989,$
 $(12, \text{class}) \mapsto 2,$
 $(20, \text{since}) \mapsto 2011,$
 $(21, \text{since}) \mapsto 1990,$
 $(21, \text{status}) \mapsto \text{'self-employed'},$
 $(22, \text{since}) \mapsto 2006$

Contemporary Graph Modeling

The PGM incorporates four basic traits of contemporary graph data models, namely

- direction,
- multi-graph,
- labels, and
- properties,

in one unifying model.

Direction

- A property graph is a directed graph
- The PGM defines edges as ordered pairs of vertices
- Hence, edges have a direction

Direction

- A property graph is a directed graph
 - The PGM defines edges as ordered pairs of vertices
 - Hence, edges have a direction
-
- Undirected edges define edges as 2-element subsets of the set of vertices
 - The codomain of η is $\text{codom}(\eta) = \{\{x, y\} \mid x, y \in V \wedge x \neq y\}$

Direction

- A property graph is a directed graph
 - The PGM defines edges as ordered pairs of vertices
 - Hence, edges have a direction
-
- Undirected edges define edges as 2-element subsets of the set of vertices
 - The codomain of η is $\text{codom}(\eta) = \{\{x, y\} \mid x, y \in V \wedge x \neq y\}$
-
- Some practical graph models are “mixed” with undirected and directed edges

Direction

- A property graph is a directed graph
 - The PGM defines edges as ordered pairs of vertices
 - Hence, edges have a direction
-
- Undirected edges define edges as 2-element subsets of the set of vertices
 - The codomain of η is $\text{codom}(\eta) = \{\{x, y\} \mid x, y \in V \wedge x \neq y\}$
-
- Some practical graph models are “mixed” with undirected and directed edges
-
- Represent undirected edge $\{x, y\}$ by directed edges (x, y) and (y, x)
 - In this sense, directed graphs are a generalization of undirected graphs

Multi-Graph

- A property graph is a multi-graph, that is, the same ordered pair of vertices may have multiple edges between them

Multi-Graph

- A property graph is a multi-graph, that is, the same ordered pair of vertices may have multiple edges between them
- Simple graphs (in contrast to multi-graphs) additionally require $\eta : E \rightarrow V \times V$ to be injective (one-to-one), that is, $\eta(e_1) = \eta(e_2)$ implies that $e_1 = e_2$

Multi-Graph

- A property graph is a multi-graph, that is, the same ordered pair of vertices may have multiple edges between them
- Simple graphs (in contrast to multi-graphs) additionally require $\eta : E \rightarrow V \times V$ to be injective (one-to-one), that is, $\eta(e_1) = \eta(e_2)$ implies that $e_1 = e_2$
- Multi-Graphs are a generalization of simple graphs

Labels

- A property graph is a multi-labeled graph
- The PGM allows vertices and edges to be tagged with zero or more labels

Labels

- A property graph is a multi-labeled graph
- The PGM allows vertices and edges to be tagged with zero or more labels

$$\lambda : V \cup E \rightarrow \mathcal{P}(\mathcal{L})$$

- A graph model may require exactly one label per vertex and edge, i.e., $\text{codom}(\lambda) = \mathcal{L}$
- Some graph models allow labels for either vertices or edges only, i.e., $\text{dom}(\lambda) = V$ or $\text{dom}(\lambda) = E$, respectively.

Labels

- A property graph is a multi-labeled graph
- The PGM allows vertices and edges to be tagged with zero or more labels

$$\lambda : V \cup E \rightarrow \mathcal{P}(\mathcal{L})$$

- A graph model may require exactly one label per vertex and edge, i.e., $\text{codom}(\lambda) = \mathcal{L}$
 - Some graph models allow labels for either vertices or edges only, i.e., $\text{dom}(\lambda) = V$ or $\text{dom}(\lambda) = E$, respectively.
-
- Not assigning any labels turns a labelled into an unlabelled graph
 - $\forall x \in V \cup E (\lambda(x) = \emptyset)$
-
- Multi-labelled graphs are extensions for all of these variations

Properties

- A property graph is a key-value-attributed graph $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{N}$
- Enriching vertices and edges with data in the form of key-value pairs

Properties

- A property graph is a key-value-attributed graph $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{N}$
 - Enriching vertices and edges with data in the form of key-value pairs
-
- “Data-attributed” graph models only support vertices and edges being attributed with a chunk of binary data
 - $\nu : V \cup E \rightarrow \{0, 1\}^*$ ($\mathcal{K} = \{0\}, \mathcal{N} = \{0, 1\}^*$)

Properties

- A property graph is a key-value-attributed graph $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{N}$
 - Enriching vertices and edges with data in the form of key-value pairs
-
- “Data-attributed” graph models only support vertices and edges being attributed with a chunk of binary data
 - $\nu : V \cup E \rightarrow \{0, 1\}^*$ ($\mathcal{K} = \{0\}, \mathcal{N} = \{0, 1\}^*$)
-
- Other graph models allow to attribute either vertices or edges only
 - $\text{dom}(\nu) = V$ or $\text{dom}(\nu) = E$

Properties

- A property graph is a key-value-attributed graph $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{N}$
 - Enriching vertices and edges with data in the form of key-value pairs
-
- “Data-attributed” graph models only support vertices and edges being attributed with a chunk of binary data
 - $\nu : V \cup E \rightarrow \{0, 1\}^*$ ($\mathcal{K} = \{0\}, \mathcal{N} = \{0, 1\}^*$)
-
- Other graph models allow to attribute either vertices or edges only
 - $\text{dom}(\nu) = V$ or $\text{dom}(\nu) = E$

Key-value attributed graphs are a generalization of data-attributed graphs as well as non-attributed graphs

Structural Extensions

- Review some extensions of the PGM proposed in recent years
- Focus on extensions that add or enrich object types of the model
- These concern the graph structure itself

Turning Paths into Objects (Objectified Paths PGM - OPPGM)

sequence of edges

- Vertices are the first-order objects in a graph
- In multi-graphs, such as property graphs, edges are second-order objects.
- They attach data, in the form of labels and properties, to (ordered) pairs of vertices, i.e., they are statements about specific subsets of lower-order objects.

Turning Paths into Objects (Objectified Paths PGM - OPPGM)

- Vertices are the first-order objects in a graph
- In multi-graphs, such as property graphs, edges are second-order objects.
- They attach data, in the form of labels and properties, to (ordered) pairs of vertices, i.e., they are statements about specific subsets of lower-order objects.

Objectified paths can be viewed as third-order objects that attach labels and properties to sequences of edges

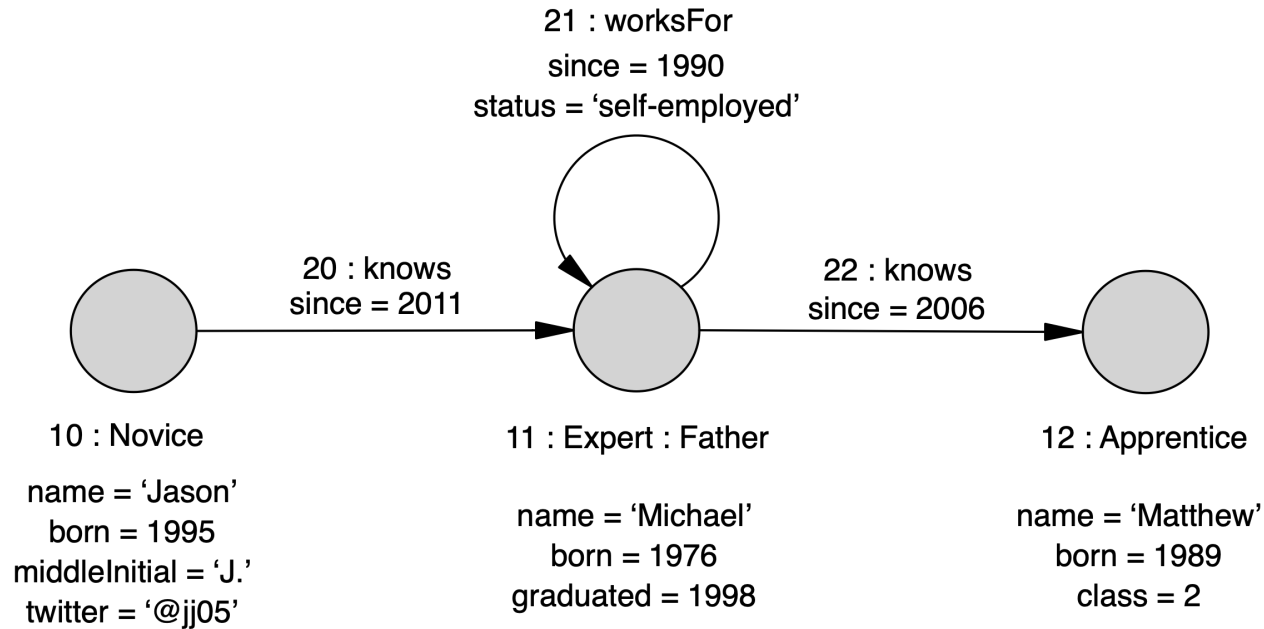
Turning Paths into Objects (Objectified Paths PGM - OPPGM)

- Vertices are the first-order objects in a graph
- In multi-graphs, such as property graphs, edges are second-order objects.
- They attach data, in the form of labels and properties, to (ordered) pairs of vertices, i.e., they are statements about specific subsets of lower-order objects.

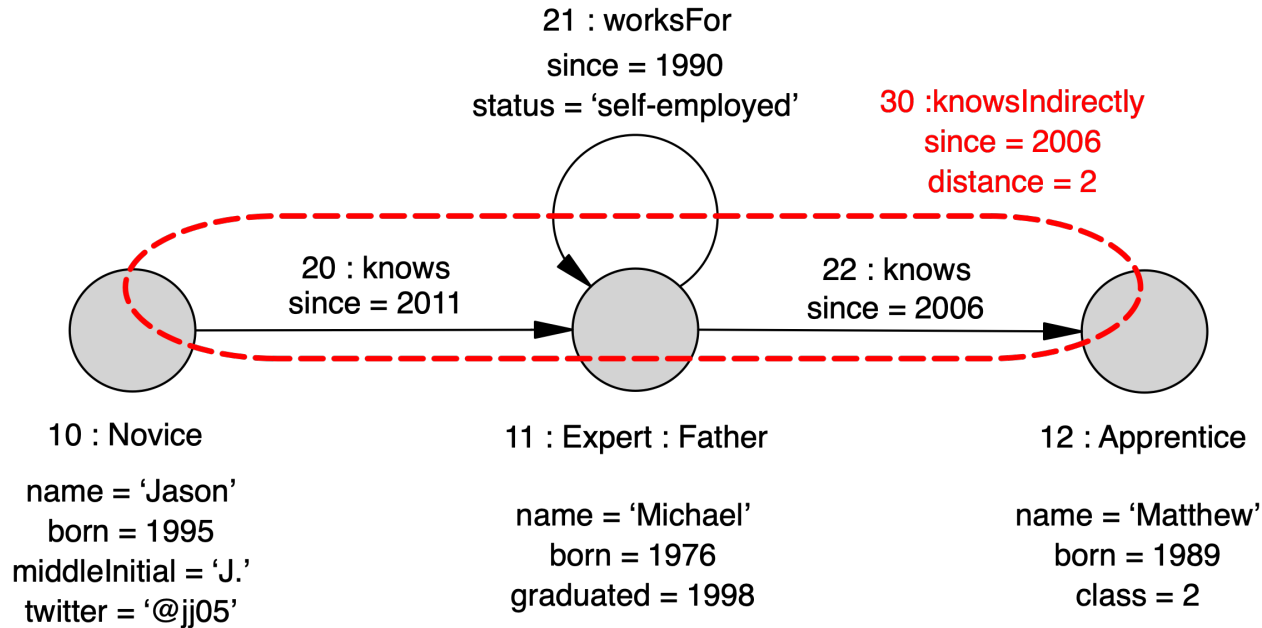
Objectified paths can be viewed as third-order objects that attach labels and properties to sequences of edges

A graph can have multiple objectified paths which can represent identical or overlapping edge sequences

Example of an OPG-Graph



Example of an OPG-Graph



Path 30

- represents edge sequence [20, 22]
- has label *:knowsIndirectly*
- has two properties *since = 2006* and *distance = 2*

Objectified Paths Property Graph Model - Formal Definition

- P is a set of paths

- $E^n = \underbrace{E \times \dots \times E}_{n \text{ times}}$

Objectified Paths Property Graph Model - Formal Definition

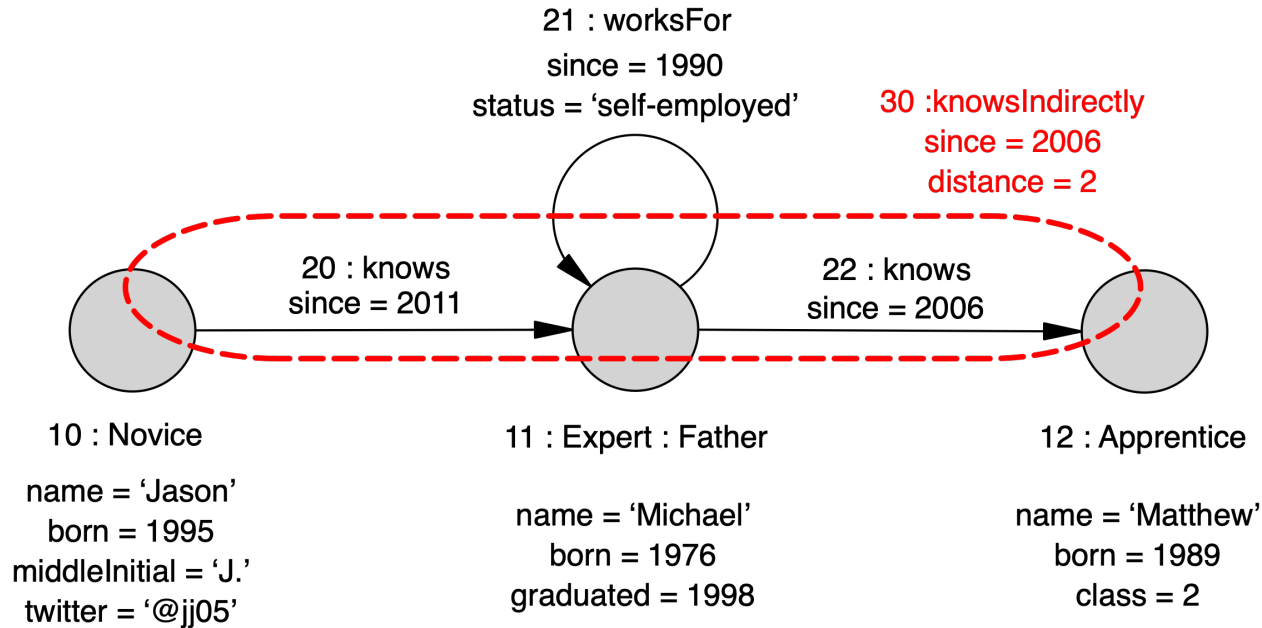
- P is a set of paths

- $E^n = \underbrace{E \times \dots \times E}_{n \text{ times}}$

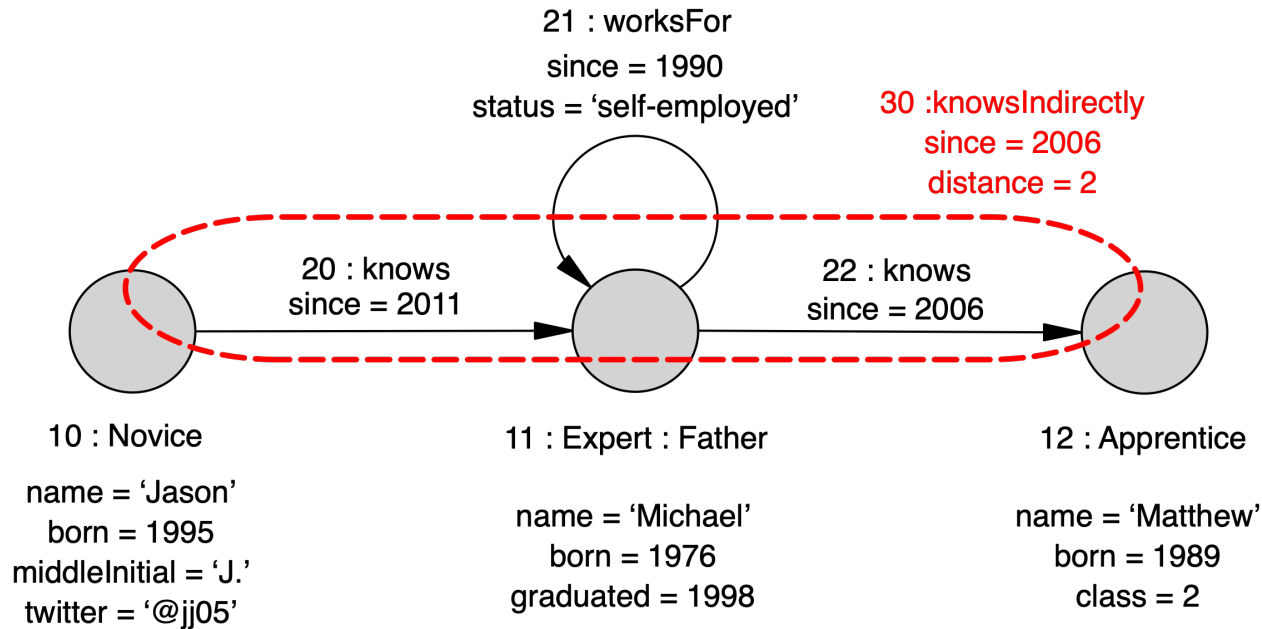
An OPPGM is a structure $(V, E, P, \eta, \delta, \lambda, \nu)$ where

- $V \subseteq \mathcal{O}$ is a finite set of objects, called *vertices*,
- $E \subseteq \mathcal{O}$ is a finite set of objects, called *edges*,
- $P \subseteq \mathcal{O}$ is a finite set of objects, called *paths*,
- $\eta : E \rightarrow V \times V$ is a function assigning to each edge an ordered pair of vertices,
- $\delta : P \rightarrow \bigcup_{n \geq 0} E^n$ is a total function assigning to every path a sequence of edges,
- $\lambda : V \cup E \cup P \rightarrow \mathcal{P}(\mathcal{L})$ is a function assigning to each object a finite set of labels, and
- $\nu : (V \cup E \cup P) \times \mathcal{K} \rightarrow \mathcal{N}$ is a partial function assigning values for properties to objects, such that δ assigns only sequences of subsequent incident edges, and the set of domain values where ν is defined is finite.

Example of Formal Definition for OPPG-Graph



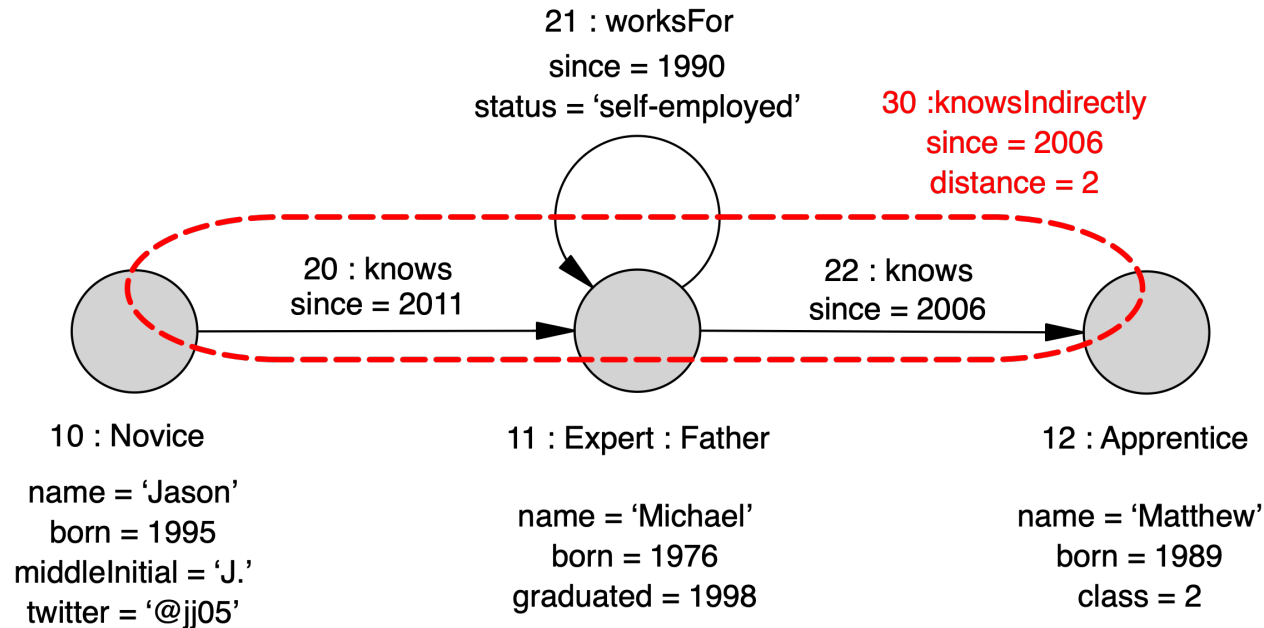
Example of Formal Definition for OPPG-Graph



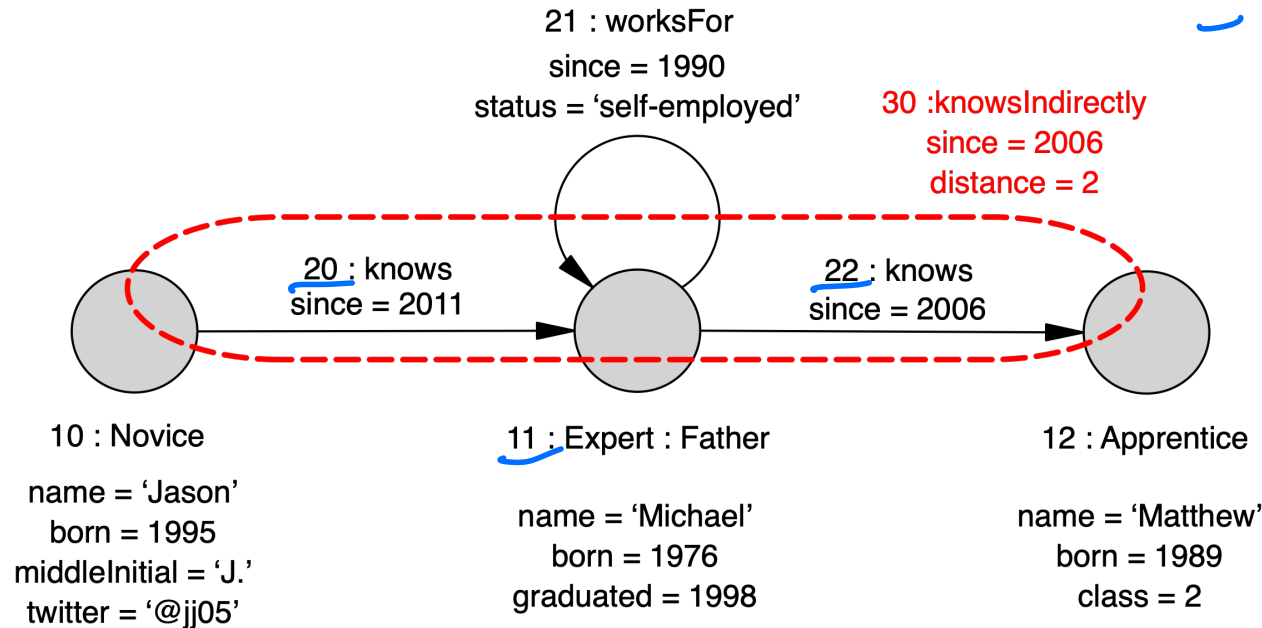
$\delta(30)$

[20, 22]

Example of Formal Definition for OPPG-Graph

 $\delta(30)$ $[20, 22]$ $\lambda(30)$ *knowsIndirectly*

Example of Formal Definition for OPPG-Graph



$\delta(30)$

[20, 22] with.

$\text{trg}(20) = 11 = \text{Src}(22)$

$\lambda(30)$

knowsIndirectly

ν

- $(30, \text{since}) \mapsto 2006$
- $(30, \text{distance}) \mapsto 2$

Turning Subgraphs into Objects (Objectified Subgraph PGM - OSPGM)

- Objectified subgraphs are higher-order objects
- They allow us to make statements about specific subsets of vertices and edges that constitute a graph themselves

Turning Subgraphs into Objects (Objectified Subgraph PGM - OSPGM)

- Objectified subgraphs are higher-order objects
 - They allow us to make statements about specific subsets of vertices and edges that constitute a graph themselves
-
- Objectified subgraphs of a base graph may overlap
 - Their union may also be a subset of the base graph
 - Each individual objectified subgraph must have at least one vertex

Turning Subgraphs into Objects (Objectified Subgraph PGM - OSPGM)

Subgraph.

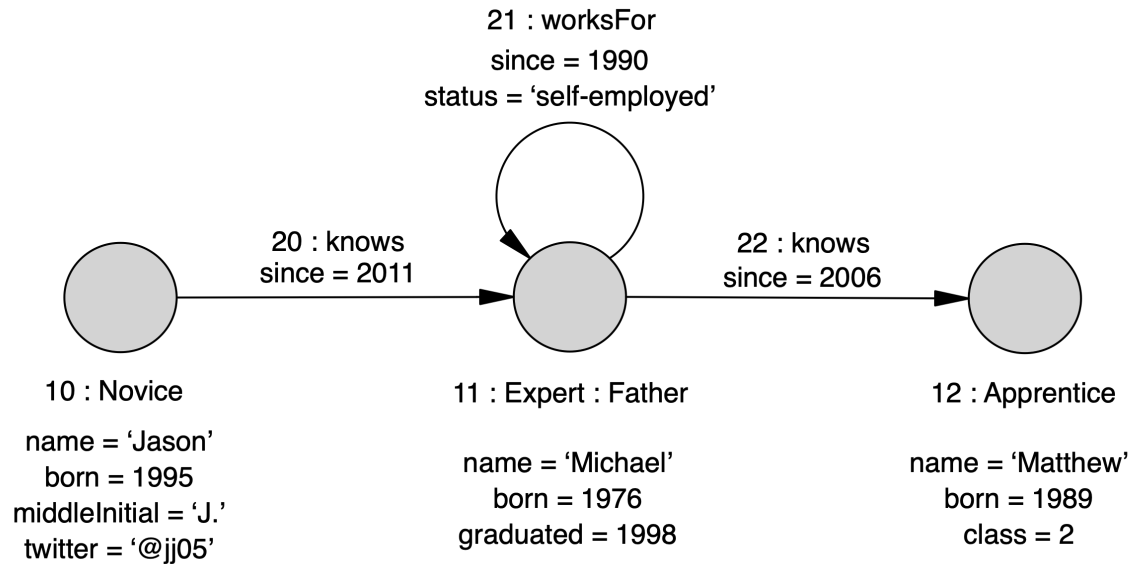
- Objectified subgraphs are higher-order objects
- They allow us to make statements about specific subsets of vertices and edges that constitute a graph themselves

include source

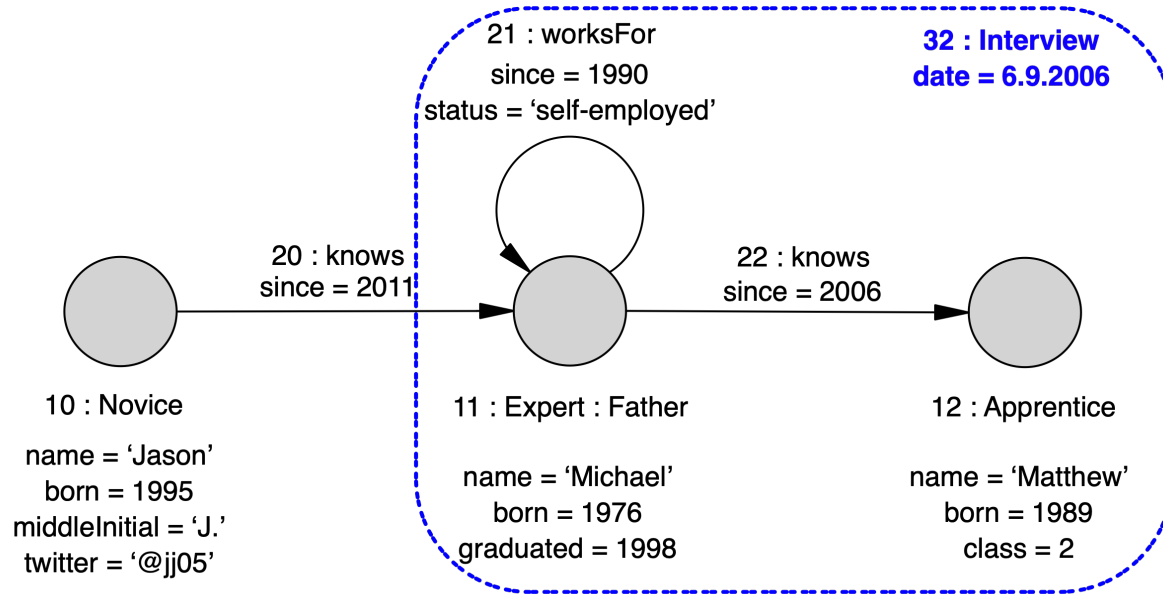
- Objectified subgraphs of a base graph may overlap
- Their union may also be a subset of the base graph
- Each individual objectified subgraph must have at least one vertex

- An OSPGM graph can have an empty set of subgraphs
- Hence, OSPGM is a strict generalization of PGM

Example of an OSPG-Graph



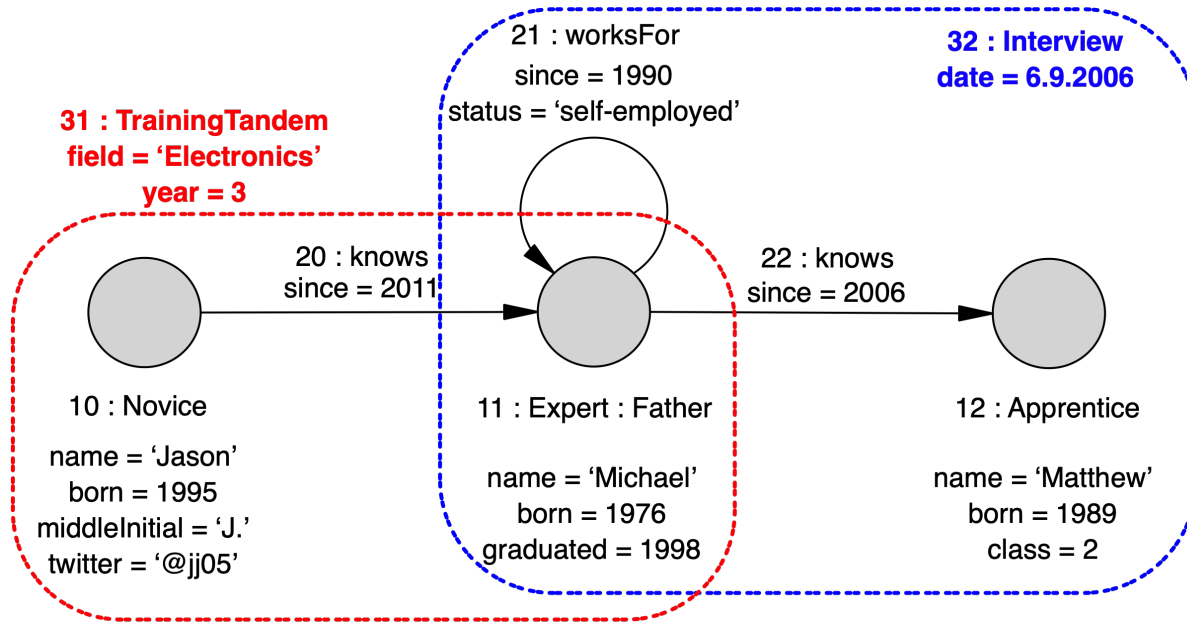
Example of an OSPG-Graph



Subgraph 32

- has vertices 11 and 12
- has edges 21 and 22
- has label *Interview*
- has property
date = 6.9.2006

Example of an OSPG-Graph



Subgraph 31

- has vertices 10 and 11
- has edge 20
- has label *TrainingTandem*
- has properties
field = 'Electronics' and
year = 3

Subgraph 32

- has vertices 11 and 12
- has edges 21 and 22
- has label *Interview*
- has property
date = 6.9.2006

Objectified Subgraph Property Graph Model - Formal Definition

- Subgraph $g \in G$ is *well-formed* iff all edges of g are adjacent to vertices of g
- That is, for every edge $e \in \pi_2(\gamma(g))$ we have $\eta(e) \in \pi_1(\gamma(g)) \times \pi_1(\gamma(g))$

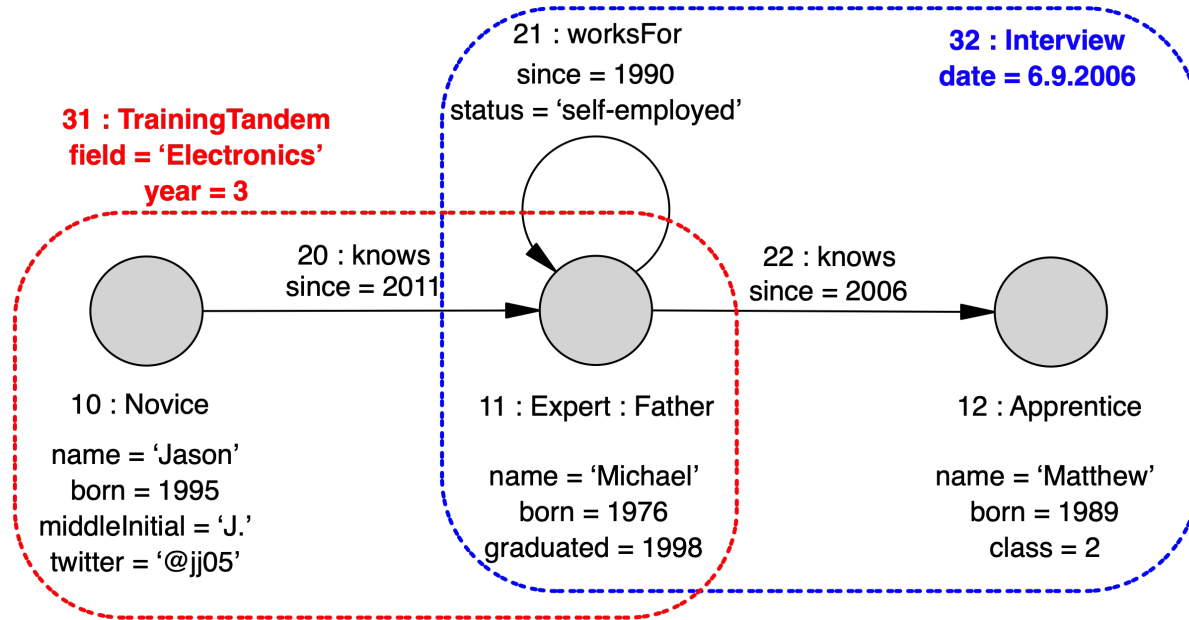
Objectified Subgraph Property Graph Model - Formal Definition

- Subgraph $g \in G$ is *well-formed* iff all edges of g are adjacent to vertices of g
- That is, for every edge $e \in \pi_2(\gamma(g))$ we have $\eta(e) \in \pi_1(\gamma(g)) \times \pi_1(\gamma(g))$

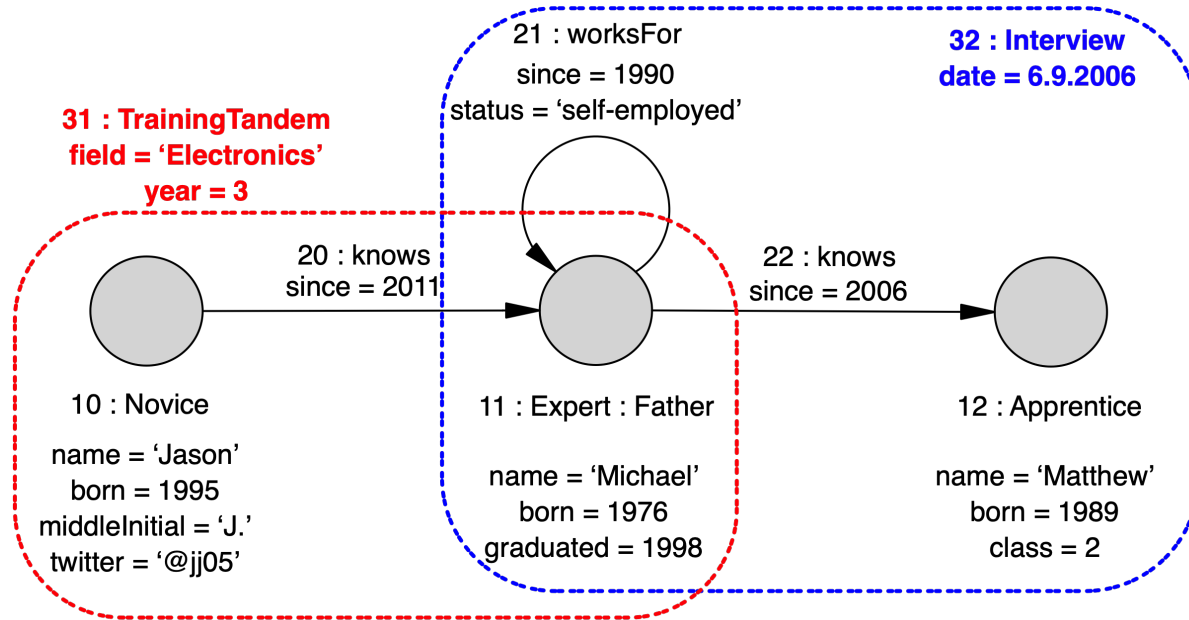
An OSPGM is a structure $(V, E, G, \eta, \gamma, \lambda, \nu)$ where

- $V \subseteq \mathcal{O}$ is a finite set of objects, called *vertices*,
- $E \subseteq \mathcal{O}$ is a finite set of objects, called *edges*,
- $G \subseteq \mathcal{O}$ is a finite set of objects, called *subgraphs*,
- $\eta : E \rightarrow V \times V$ is a function assigning to each edge an ordered pair of vertices,
- $\gamma : G \rightarrow \mathcal{P}(V) \times \mathcal{P}(E)$ is a total function assigning to every subgraph a pair of vertex set and edge set, 顶点集 边集
- $\lambda : V \cup E \cup G \rightarrow \mathcal{P}(\mathcal{L})$ is a function assigning to each object a finite set of labels, and
- $\nu : (V \cup E \cup G) \times \mathcal{K} \rightarrow \mathcal{N}$ is a partial function assigning values for properties to objects, such that V , E , and G are pairwise disjoint, the subgraphs $g \in G$ are well-formed, and the set of domain values where ν is defined is finite.

Example of Formal Definition for an OSPG-Graph



Example of Formal Definition for an OSPG-Graph

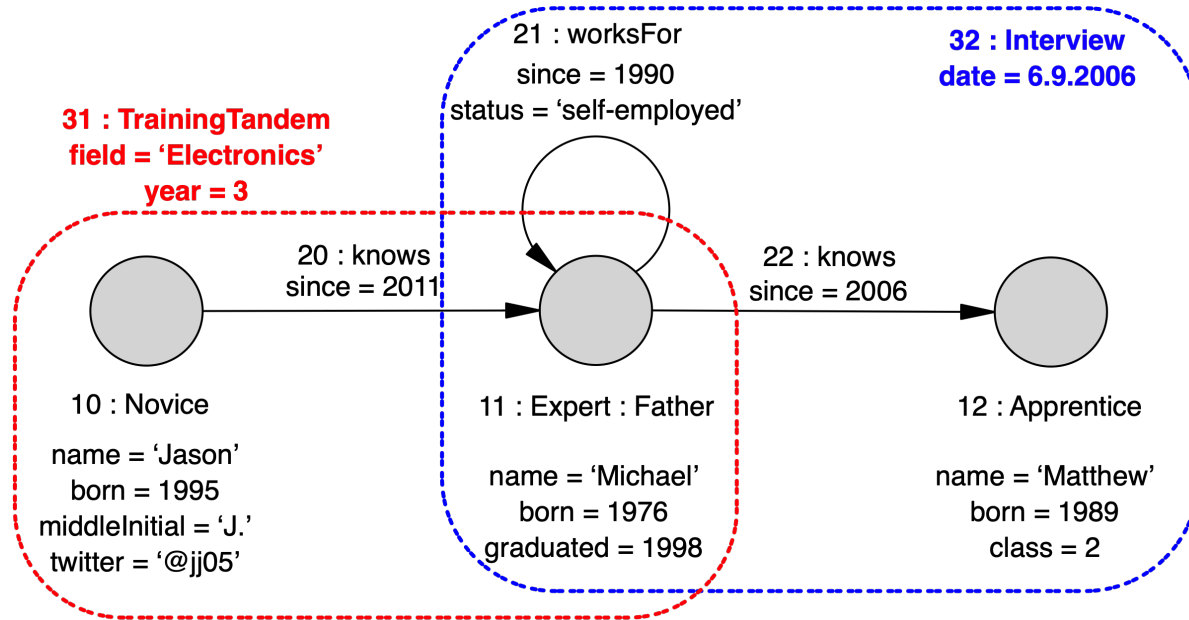


γ

- $31 \mapsto (\{10, 11\}, \{20\})$
- $32 \mapsto (\{11, 12\}, \{21, 22\})$

31 and 32 each well-formed

Example of Formal Definition for an OSPG-Graph



γ

- $31 \mapsto (\{10, 11\}, \{20\})$
- $32 \mapsto (\{11, 12\}, \{21, 22\})$

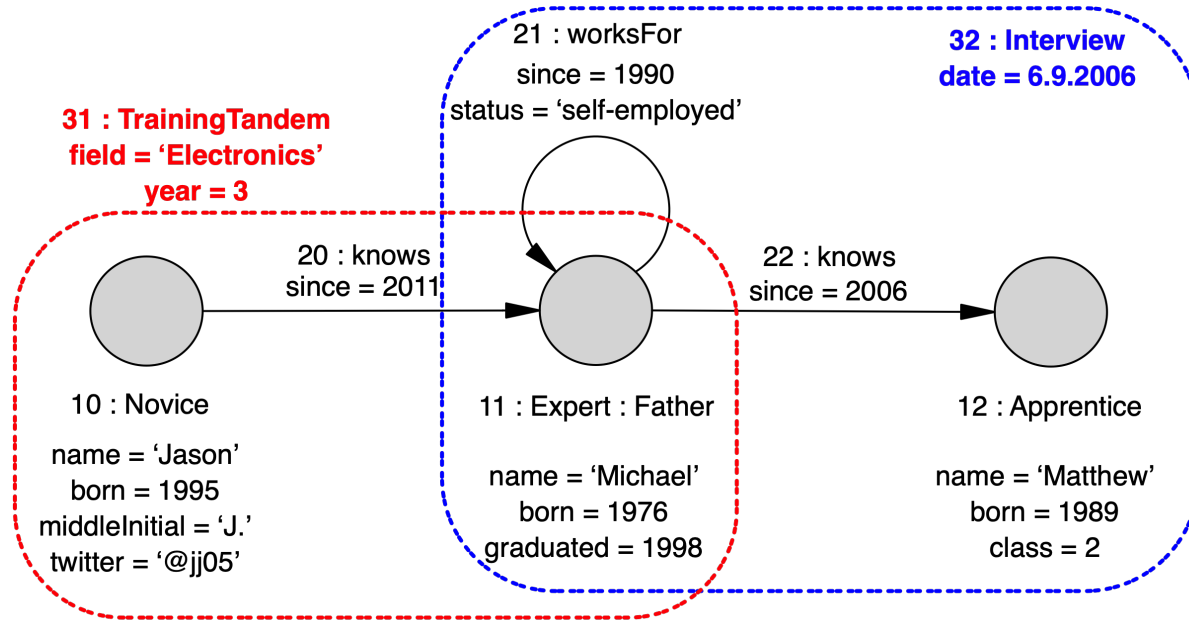
31 and 32 each well-formed

λ

labels

- $31 \mapsto \{ \textit{TrainingTandem} \}$
- $32 \mapsto \{ \textit{Interview} \}$

Example of Formal Definition for an OSPG-Graph



γ

- $31 \mapsto (\{10, 11\}, \{20\})$
- $32 \mapsto (\{11, 12\}, \{21, 22\})$

31 and 32 each well-formed

λ

- $31 \mapsto \{ \textit{TrainingTandem} \}$
- $32 \mapsto \{ \textit{Interview} \}$

ν

- $(31, \textit{field}) \mapsto \textit{'Electronics'}$
- $(31, \textit{year}) \mapsto 3$
- $(32, \textit{date}) \mapsto 6.9.2006$

Turning Subgraphs into Vertices (Hypervertex PGM - HVPGM)

Permit objectified subgraphs as vertices

Turning Subgraphs into Vertices (Hypervertex PGM - HVPGM)

Permit objectified subgraphs as vertices

- Hypervertices denoting empty subgraphs represent vertices in the traditional sense
- Hypervertices denoting non-empty subgraph represent higher-order objects
- Hypervertices can be linked with edges
- HVPGM can link subgraphs to other subgraphs and or hypervertices they contain

Turning Subgraphs into Vertices (Hypervertex PGM - HVPGM)

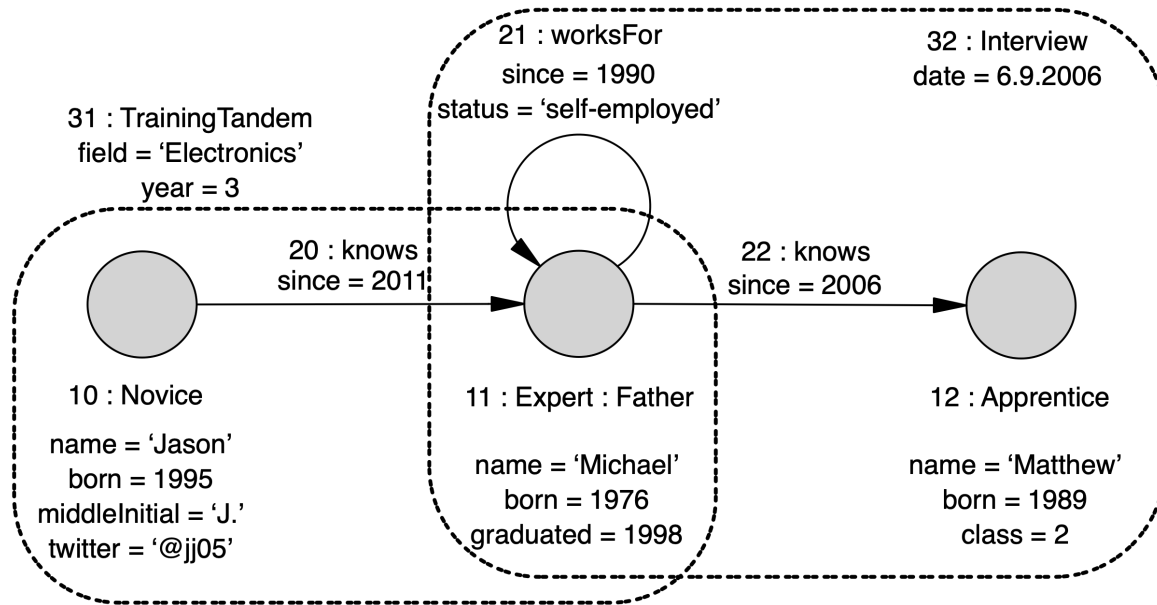
subgraph ^{当作} → vertices

Permit objectified subgraphs as vertices

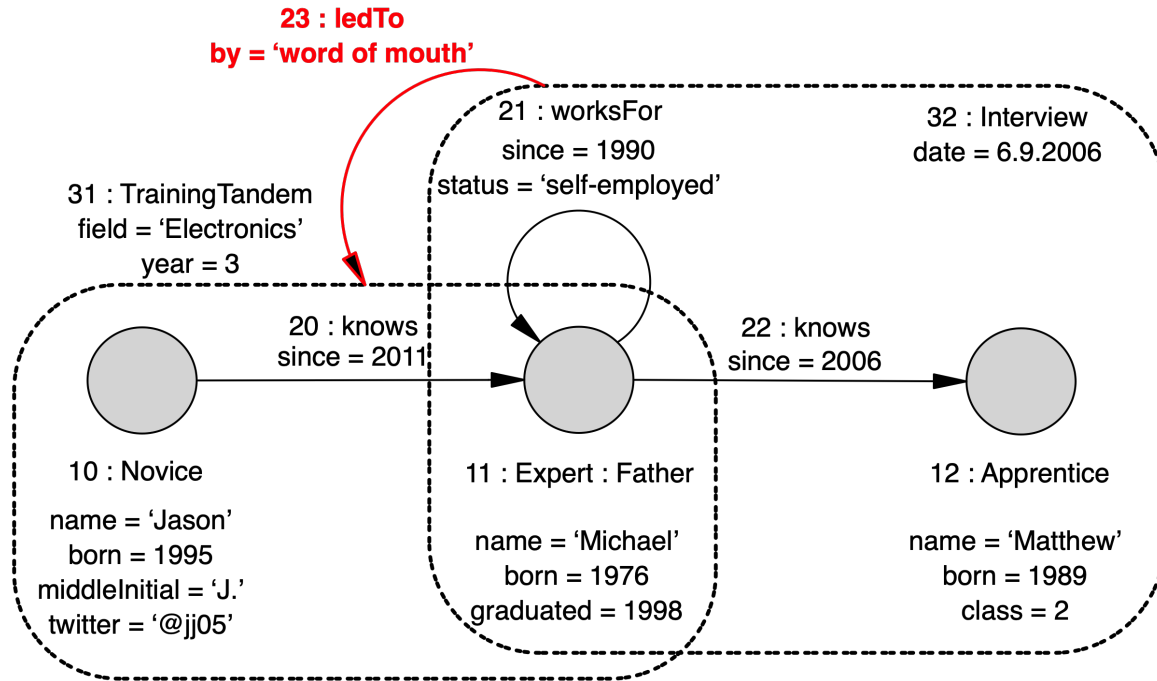
- Hypervertices denoting empty subgraphs represent vertices in the traditional sense
- Hypervertices denoting non-empty subgraph represent higher-order objects
- Hypervertices can be linked with edges
- HVPGM can link subgraphs to other subgraphs and or hypervertices they contain

- Hypervertices may have no adjacent edges
- Hence, HVPGM is a strict generalization of PGM

Example of an Hypervertex-Graph



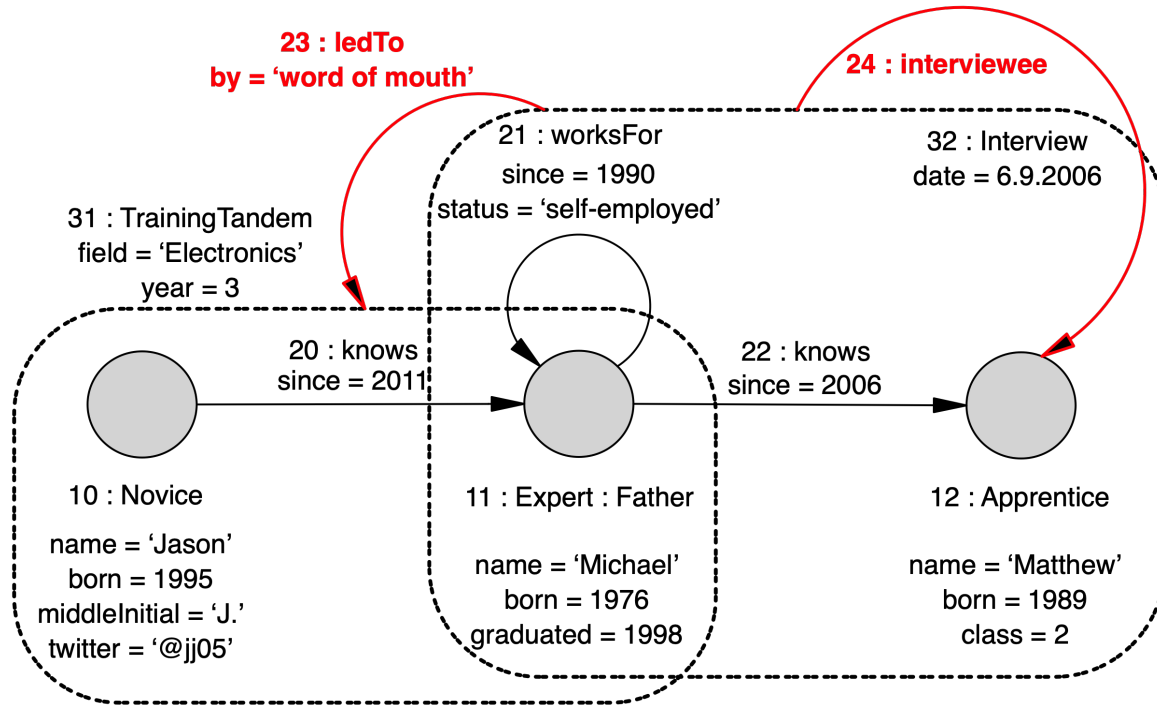
Example of an Hypervertex-Graph



Edge 23

- links hypervertex 32 to hypervertex 31
- has label *ledTo*
- has property *by = 'word of mouth'*

Example of an Hypervertex-Graph



Edge 23

- links hypervertex 32 to hypervertex 31
- has label *ledTo*
- has property *by = 'word of mouth'*

Edge 24

- links hypervertex 32 to vertex 12
- has label *Interviewee*
- has no property

Hyper Vertex Property Graph Model - Formal Definition

- Hypervertex $v \in V$ is *well-formed* iff all edges of v are adjacent to vertices of v
- That is, for every edge $e \in \pi_2(\gamma(v))$ we have $\eta(e) \in \pi_1(\gamma(v)) \times \pi_1(\gamma(v))$

Hyper Vertex Property Graph Model - Formal Definition

- Hypervertex $v \in V$ is *well-formed* iff all edges of v are adjacent to vertices of v
- That is, for every edge $e \in \pi_2(\gamma(v))$ we have $\eta(e) \in \pi_1(\gamma(v)) \times \pi_1(\gamma(v))$

An HVPGM is a structure $(V, E, \eta, \gamma, \lambda, \nu)$ where

- $V \subseteq \mathcal{O}$ is a finite set of objects, called *hypervertices*,
- $E \subseteq \mathcal{O}$ is a finite set of objects, called *edges*,
- $\eta : E \rightarrow V \times V$ is a function assigning to each edge an ordered pair of hypervertices,
- $\gamma : V \rightarrow \mathcal{P}(V) \times \mathcal{P}(E)$ is a total function assigning to every hypervertex a pair of hypervertex set and edge set,
- $\lambda : V \cup E \rightarrow \mathcal{P}(\mathcal{L})$ is a function assigning to each object a finite set of labels, and
- $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{N}$ is a partial function assigning values for properties to objects, such that V and E are pairwise disjoint, the hypervertices $v \in V$ are well-formed, and the set of domain values where ν is defined is finite.

Permit hyperedges

Hyper Edge PGM - HEPGM

Permit hyperedges

- A (hyper)edge links a non-empty sequence of vertices without repetitions

Hyper Edge PGM - HEPGM

Permit hyperedges

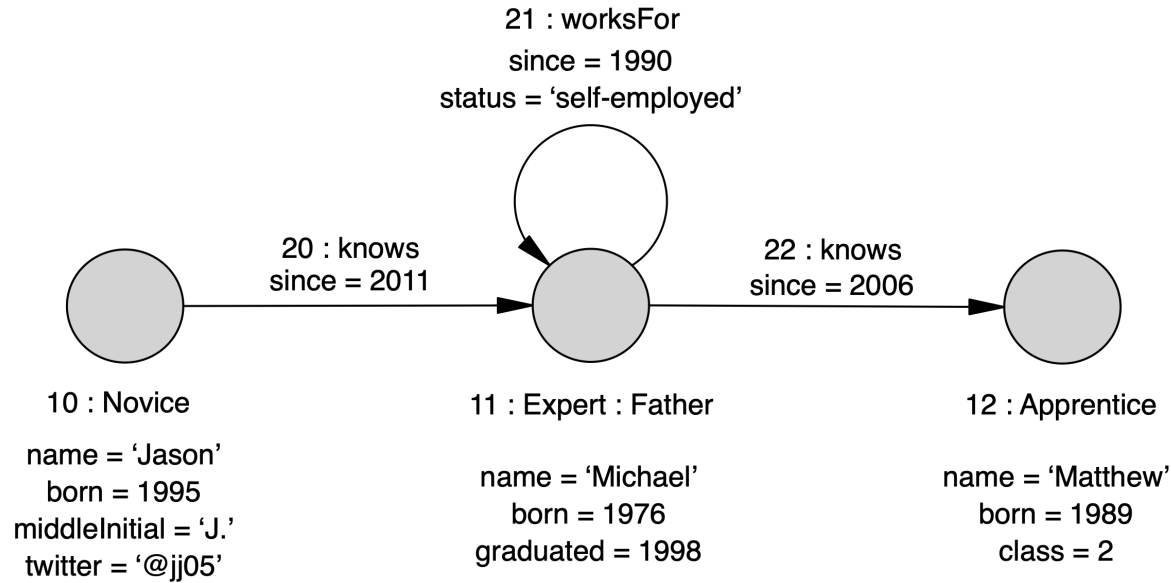
- A (hyper)edge links a non-empty sequence of vertices without repetitions
- All hyperedges may have exactly two vertices
- Hence, HEPGM is a strict generalization of PGM

Hyper Edge PGM - HEPGM

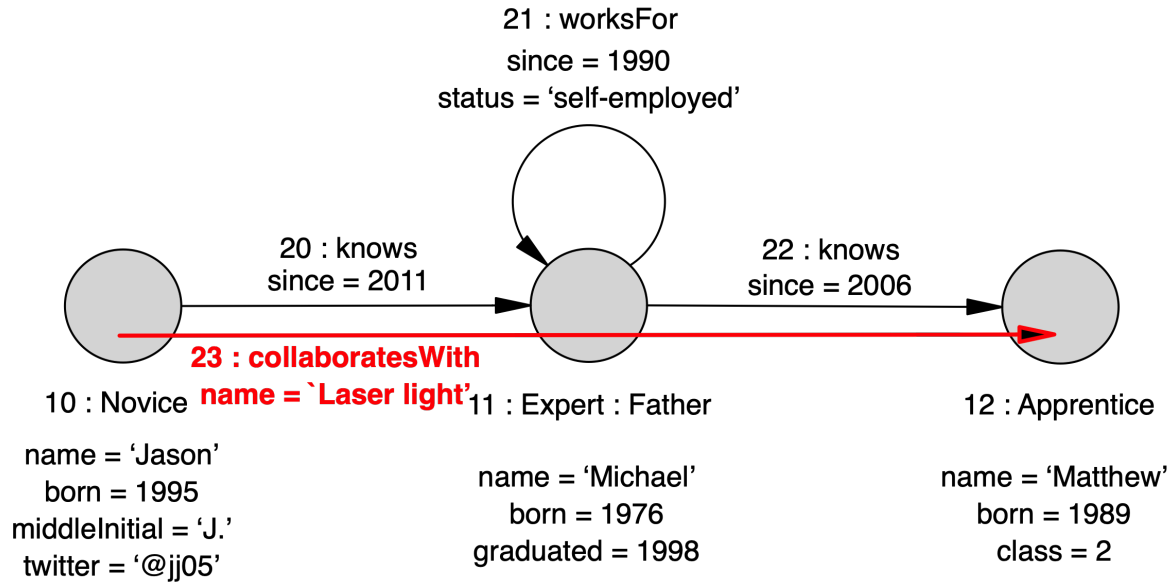
Permit hyperedges

- A (hyper)edge links a non-empty sequence of vertices without repetitions
 - All hyperedges may have exactly two vertices
 - Hence, HEPGM is a strict generalization of PGM
-
- Hyperedges represent sequences of vertices without repetitions
 - Objectified paths represent sequences of edges
 - Objectified paths may involve repetition of edges and adjacent vertices
 - Hence, hyperedges do not generalize objectified paths

Example of a Property Graph with Hyperedges



Example of a Property Graph with Hyperedges



Hyperedge 23

- connects vertices 10, 11, and 12
- has label *collaboratesWith*
- has property *name = 'Laser light'*

Hyper Edge Property Graph Model - Formal Definition

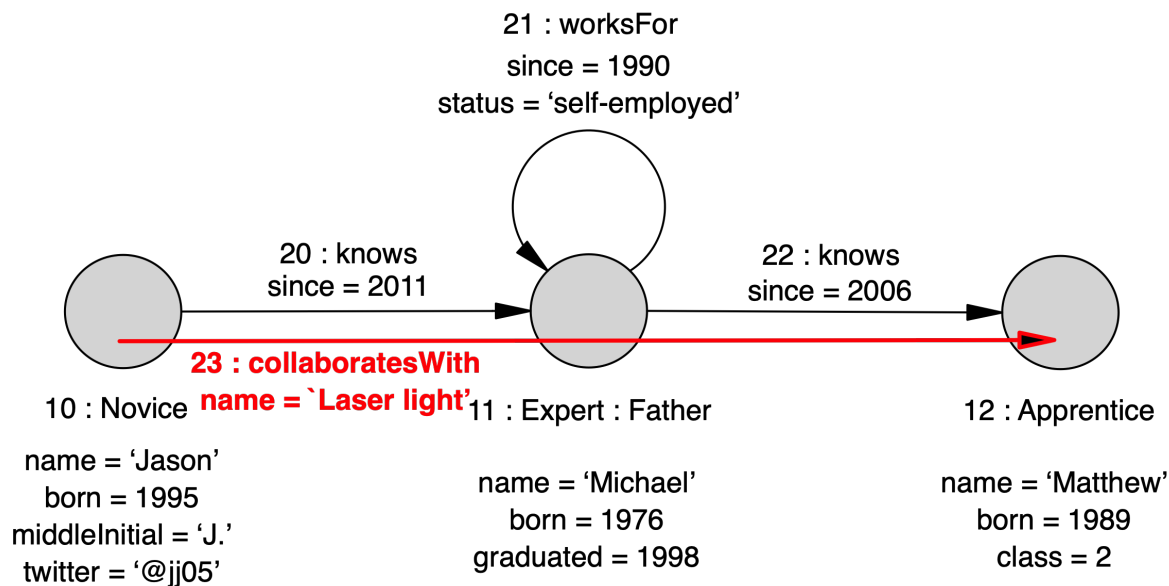


sequence of vertices

An HEPGM is a structure $(V, E, \eta, \lambda, \nu)$ where

- $V \subseteq \mathcal{O}$ is a finite set of objects, called *vertices*,
- $E \subseteq \mathcal{O}$ is a finite set of objects, called *hyperedges*,
- $\eta : E \rightarrow \bigcup_{X \in \mathcal{P}(V) - \{\emptyset\}} \{[\pi(1), \dots, \pi(|X|)] \mid \pi \text{ is a bijection from } \{1, \dots, |X|\} \text{ to } X\}$ is a function assigning to each hyperedge a non-empty sequence of vertices without repetition, 
- $\lambda : V \cup E \rightarrow \mathcal{P}(\mathcal{L})$ is a function assigning to each object a finite set of labels, and
- $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{N}$ is a partial function assigning values for properties to objects, such that V and E are pairwise disjoint, and the set of domain values where ν is defined is finite.

Formal Example of a Property Graph with Hyperedges



η

- $20 \mapsto [10, 11]$
- $21 \mapsto [11]$ *no duplication*
- $22 \mapsto [11, 12]$
- $23 \mapsto [10, 11, 12]$ *direction*

λ

- $23 \mapsto \text{collaboratesWith}$

ν

- $(23, \text{name}) \mapsto \text{'Laser light'}$

Data Representation Extensions

- Less fundamental than structural extensions
- They concern data properties, that is, value representations at the level of the objects of a graph
- Practically relevant and common in implementations
- Conceptual and theoretical work on graph querying typically neglects these extensions
- We briefly discuss the most important of such extensions

Value Types

Most implemented PGMs have typed values (e.g., integers and strings)

Value Types

Most implemented PGMs have typed values (e.g., integers and strings)

- $\mathcal{T}$ denotes finite set of value types supported by a system
- System maintains a function $\tau : \mathcal{N} \rightarrow \mathcal{T}$ that maps every value to some type

Multiple Values

- Some systems allow properties to have more than one value
- $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{N}$ needs to be generalized to $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{P}(\mathcal{N})$
- Multiple values may also be seen as the support of just another type of values, namely *sets of values*

Multiple Values

- Some systems allow properties to have more than one value
- $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{N}$ needs to be generalized to $\nu : (V \cup E) \times \mathcal{K} \rightarrow \mathcal{P}(\mathcal{N})$
- Multiple values may also be seen as the support of just another type of values, namely *sets of values*

Example

- A vertex representing a person may have a property *hobbies* with values 'Hiking', 'Climbing', 'Reading'
- $hobbies = \{ 'Hiking', 'Climbing', 'Reading' \}$

Explicit Identity

- Many systems enforce an explicit identity of the objects
- Let $\mathcal{I}$ be a set of identities used by a system
- The system enforces a bijection $\iota : \mathcal{O} \rightarrow \mathcal{I}$, which assigns a unique identity to each object

Explicit Identity

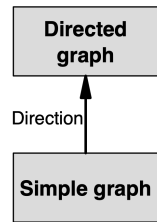
- Many systems enforce an explicit identity of the objects
- Let $\mathcal{I}$ be a set of identities used by a system
- The system enforces a bijection $\iota : \mathcal{O} \rightarrow \mathcal{I}$, which assigns a unique identity to each object

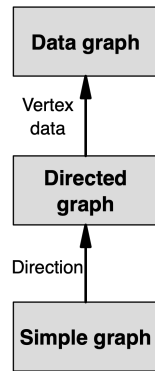
Example

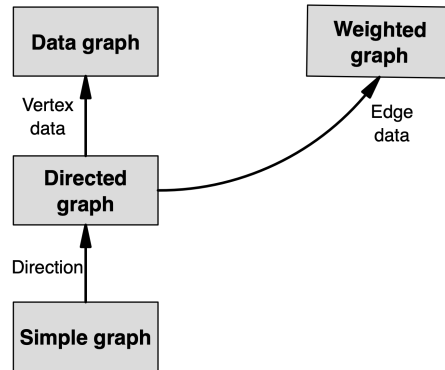
Typically, the domain of a standard data type such as Integer or Long is used as $\mathcal{I}$

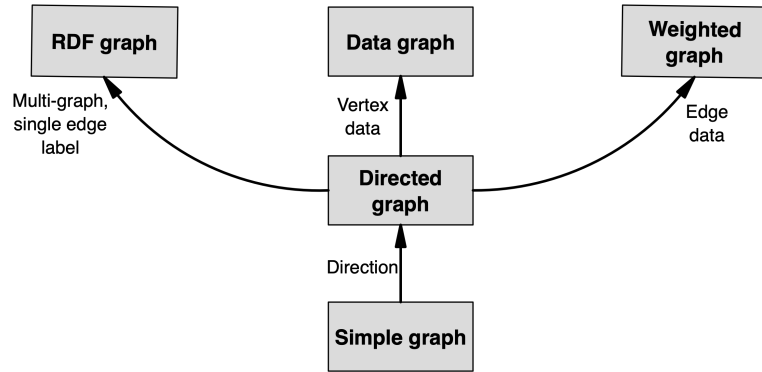
Graph model landscape

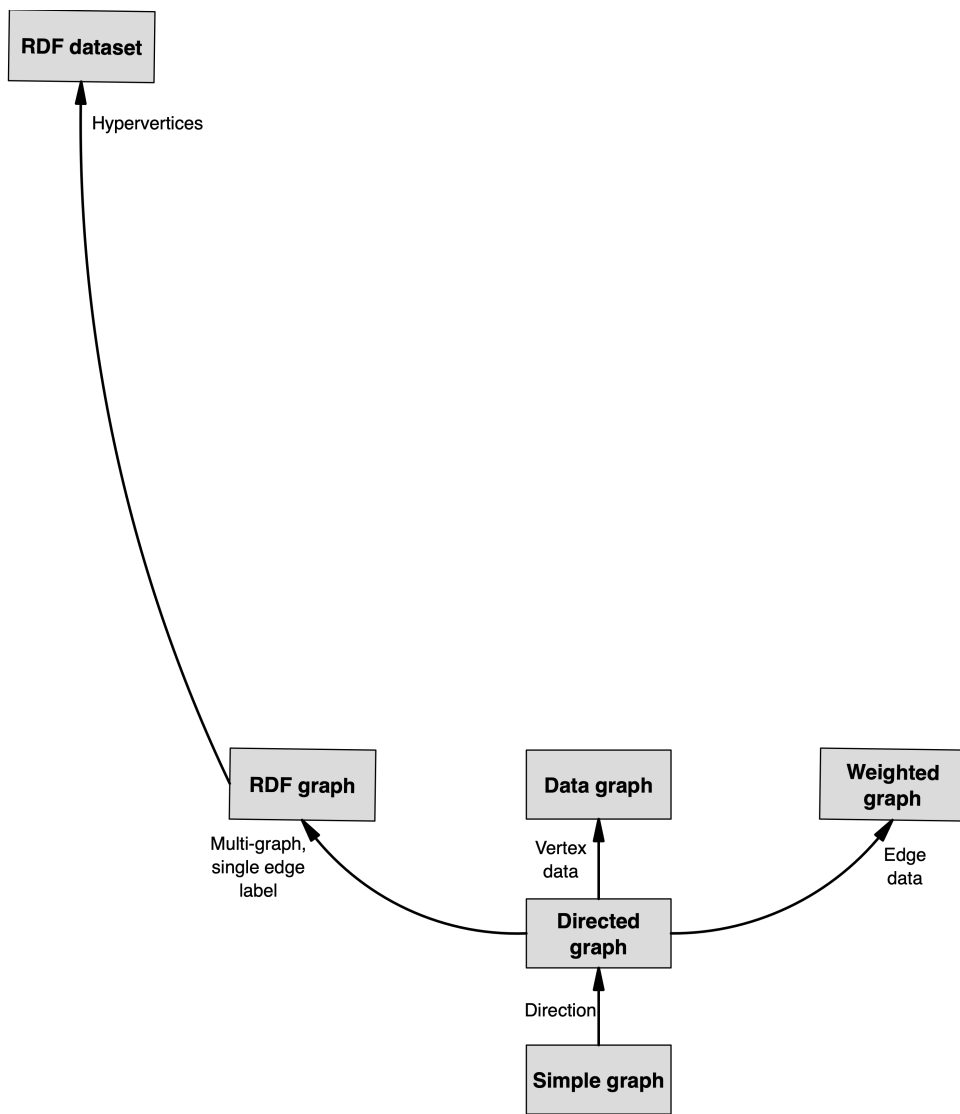
Simple graph

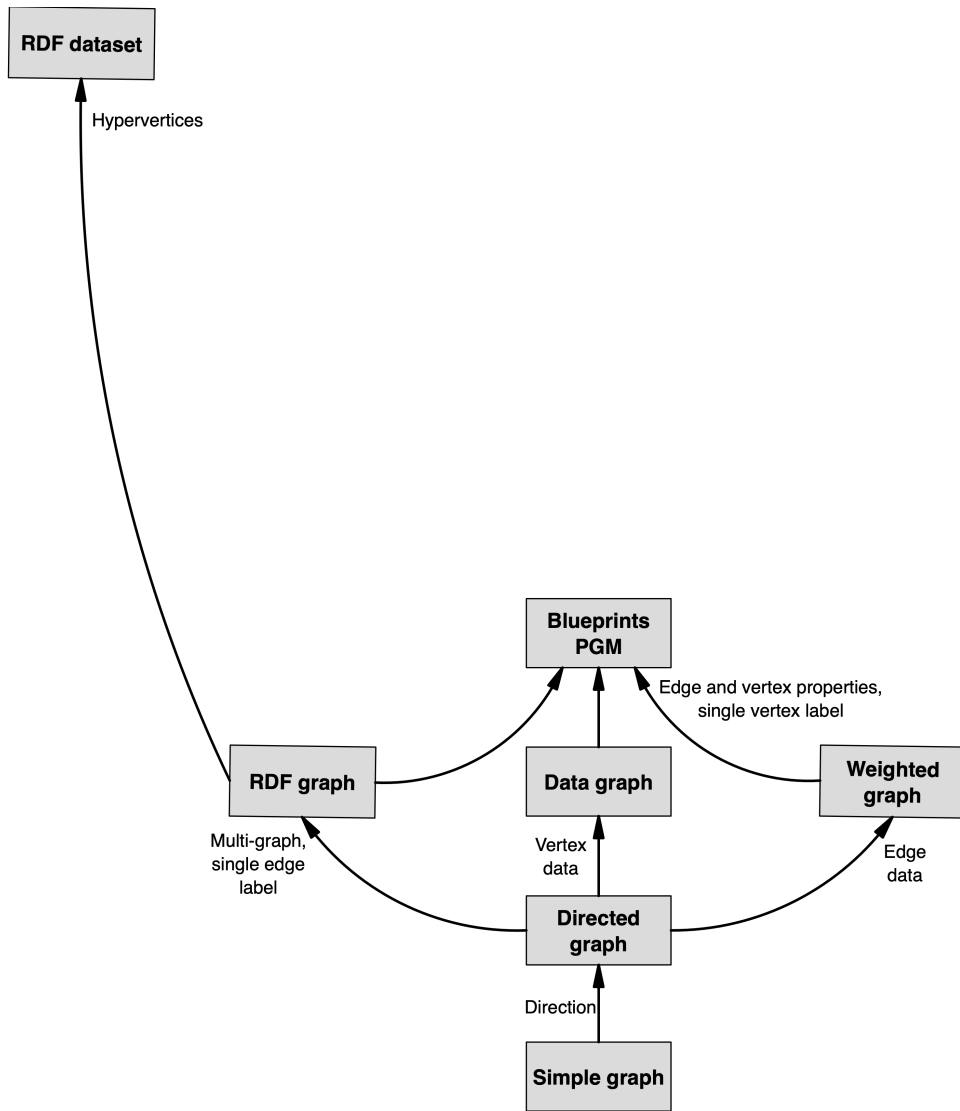


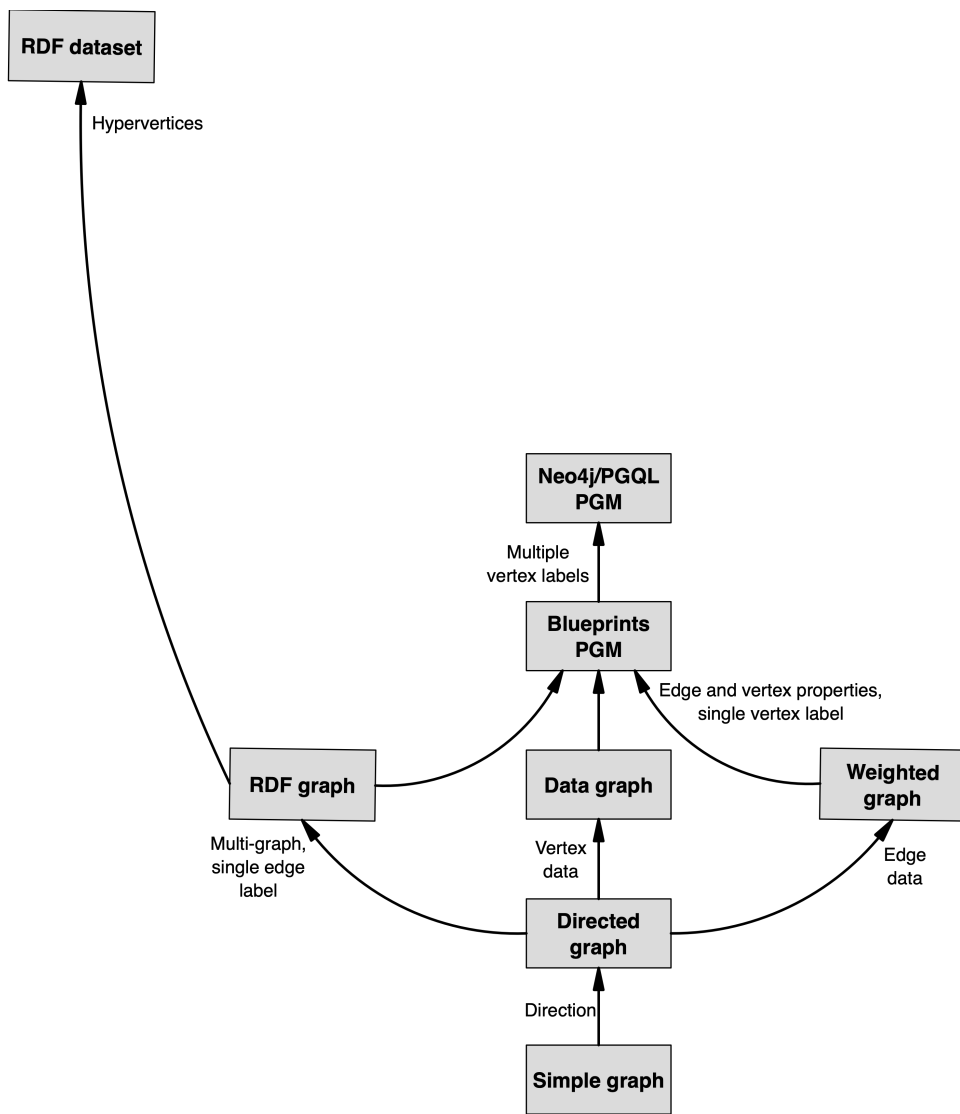


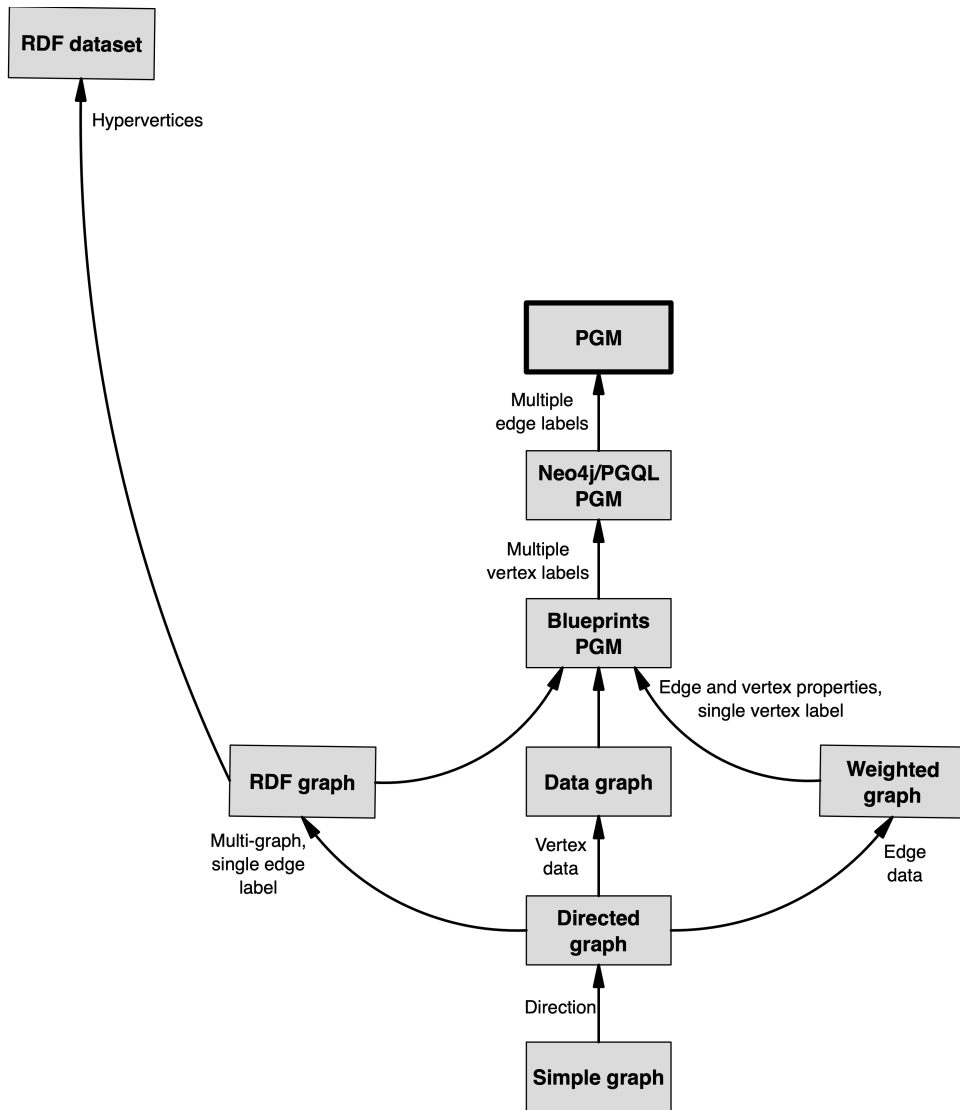


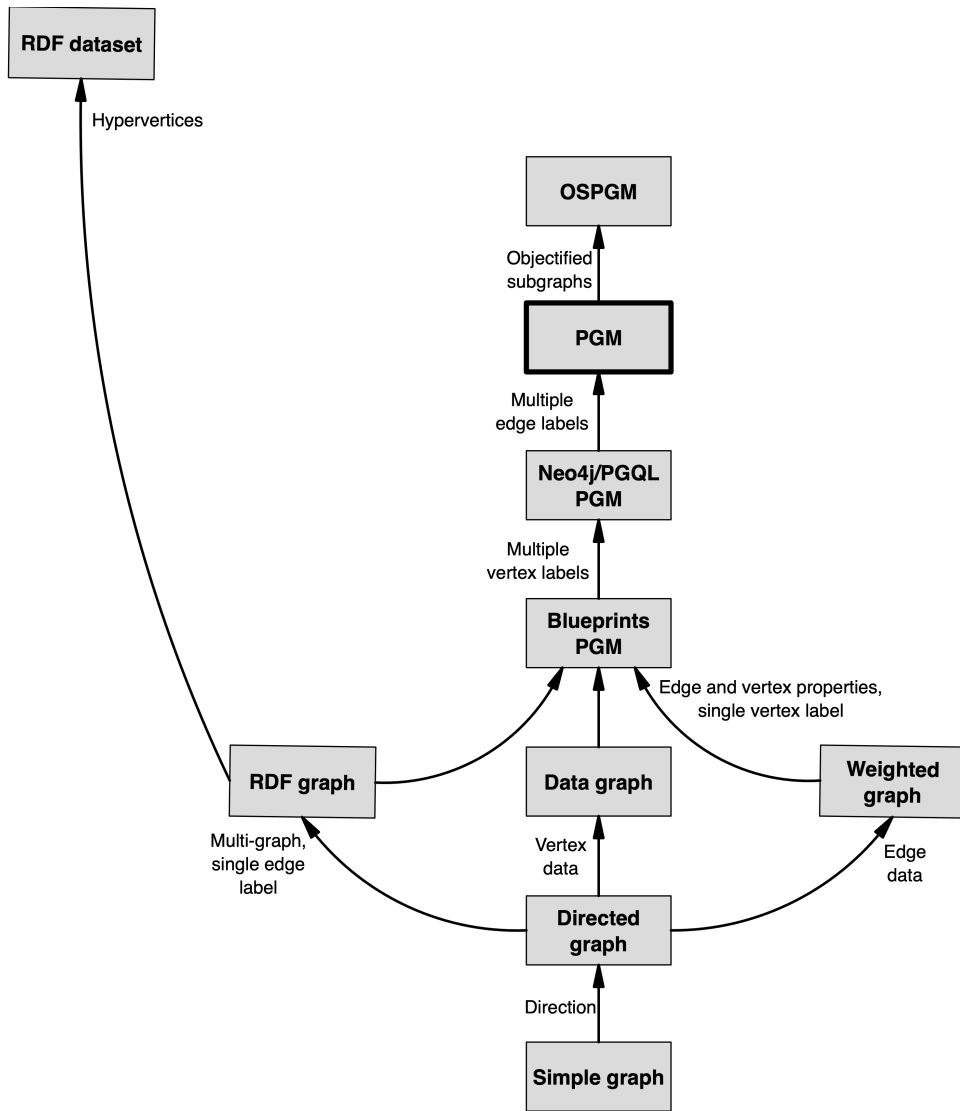


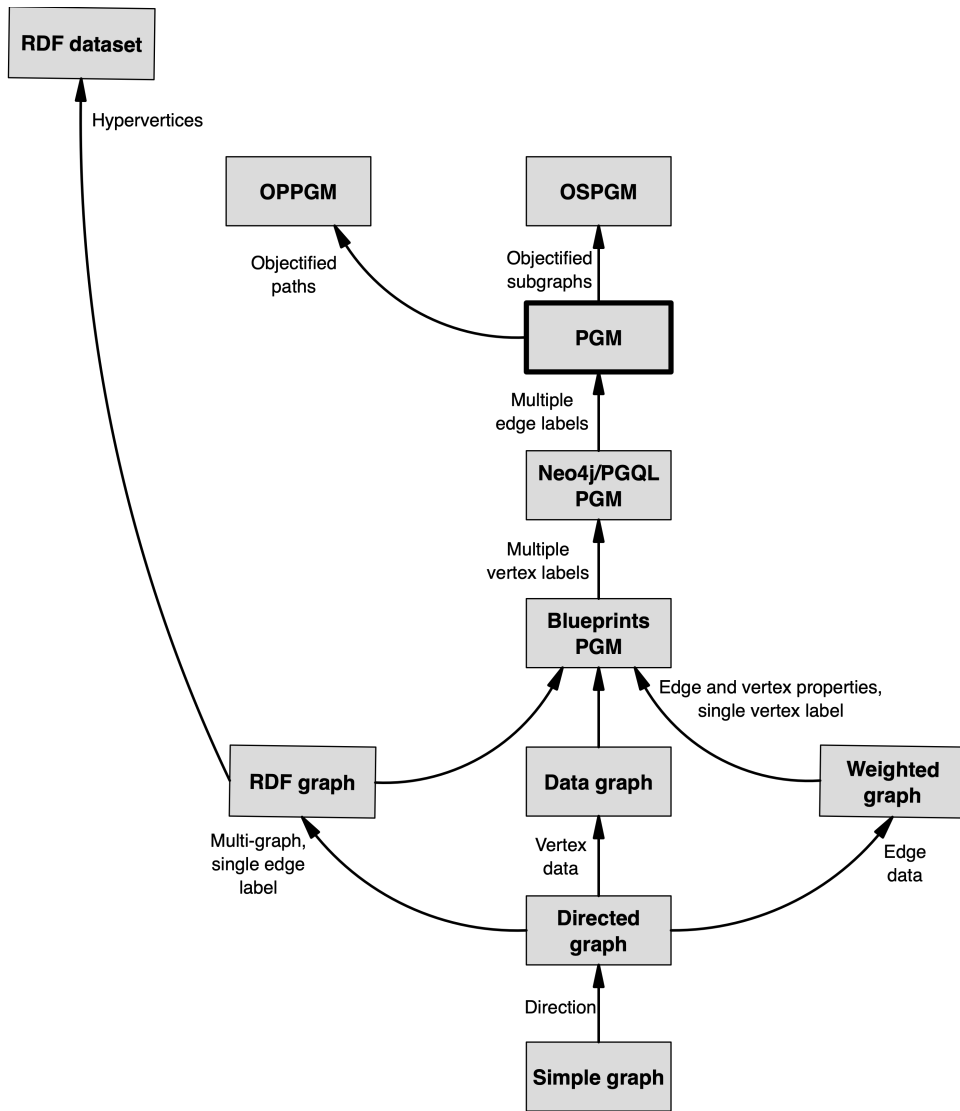


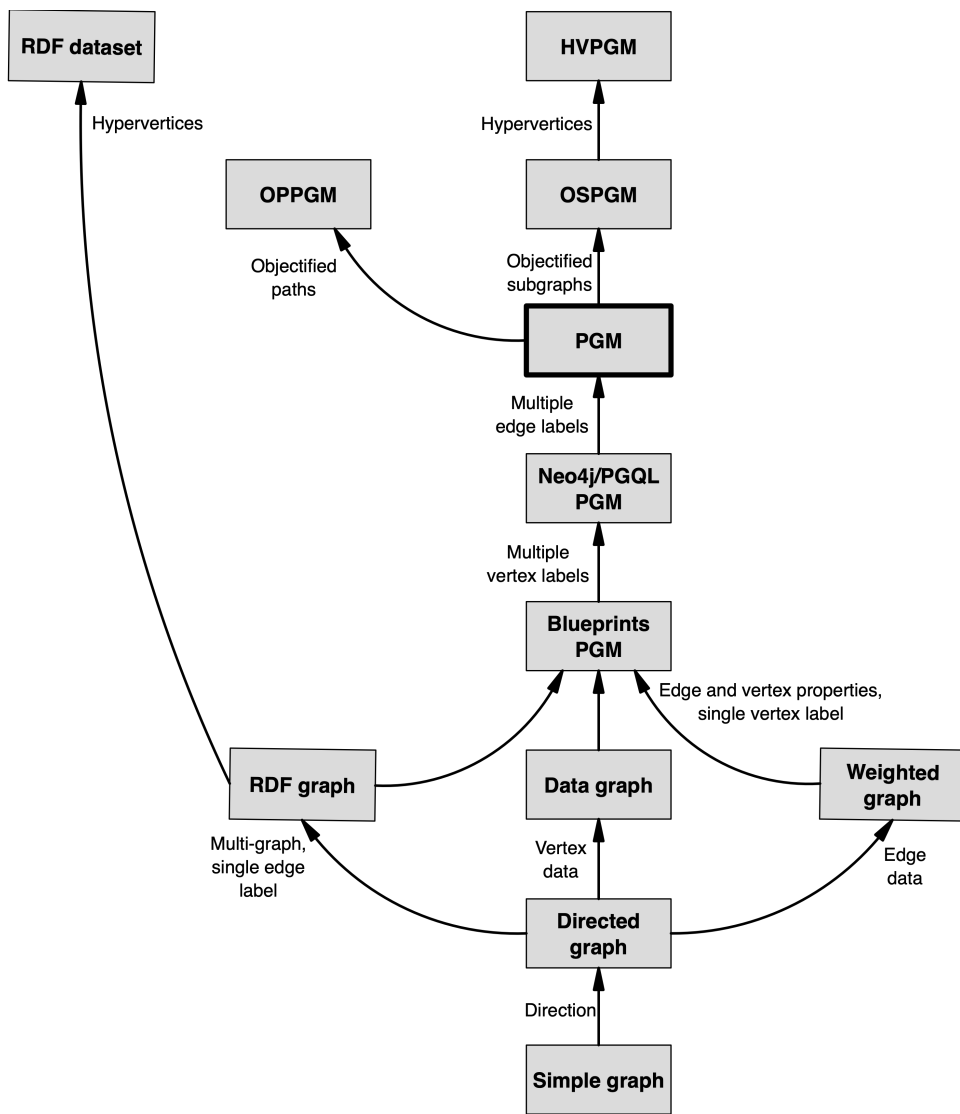


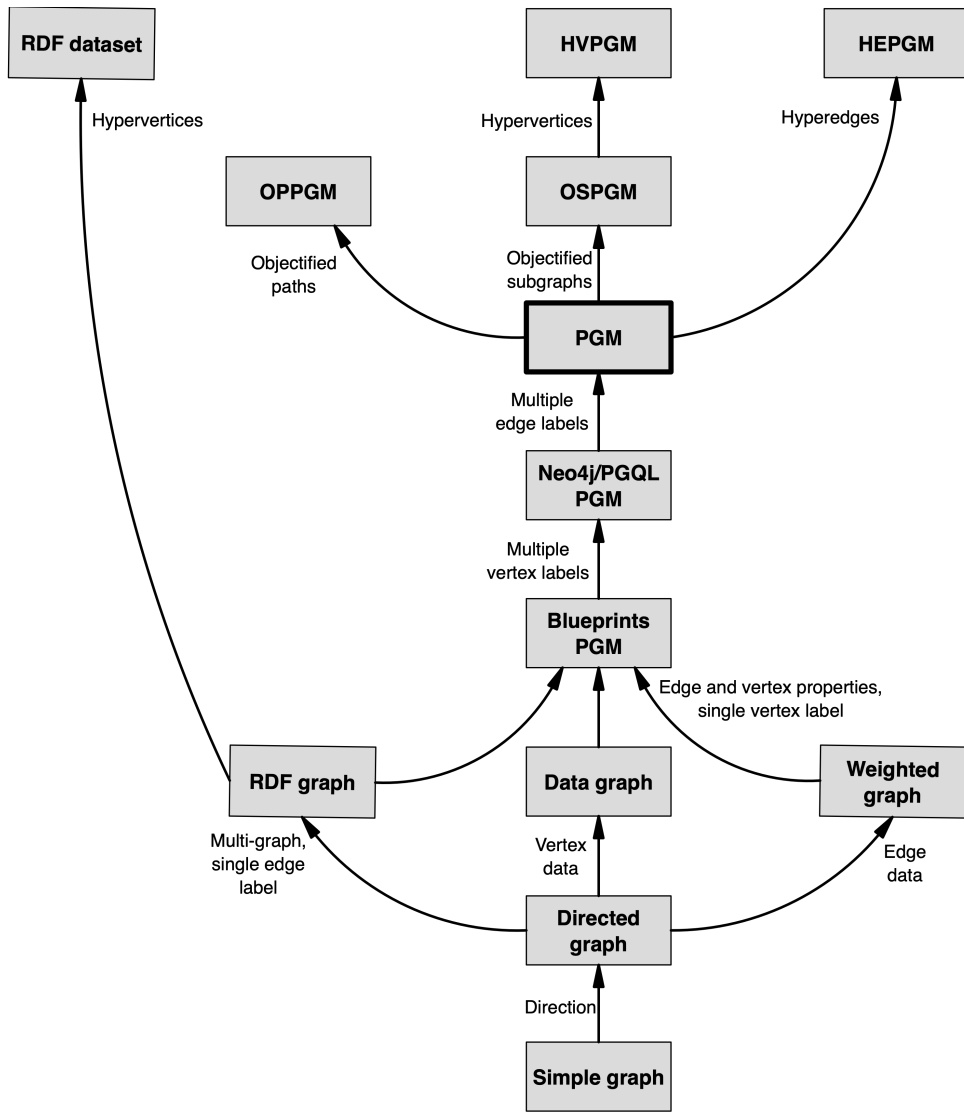


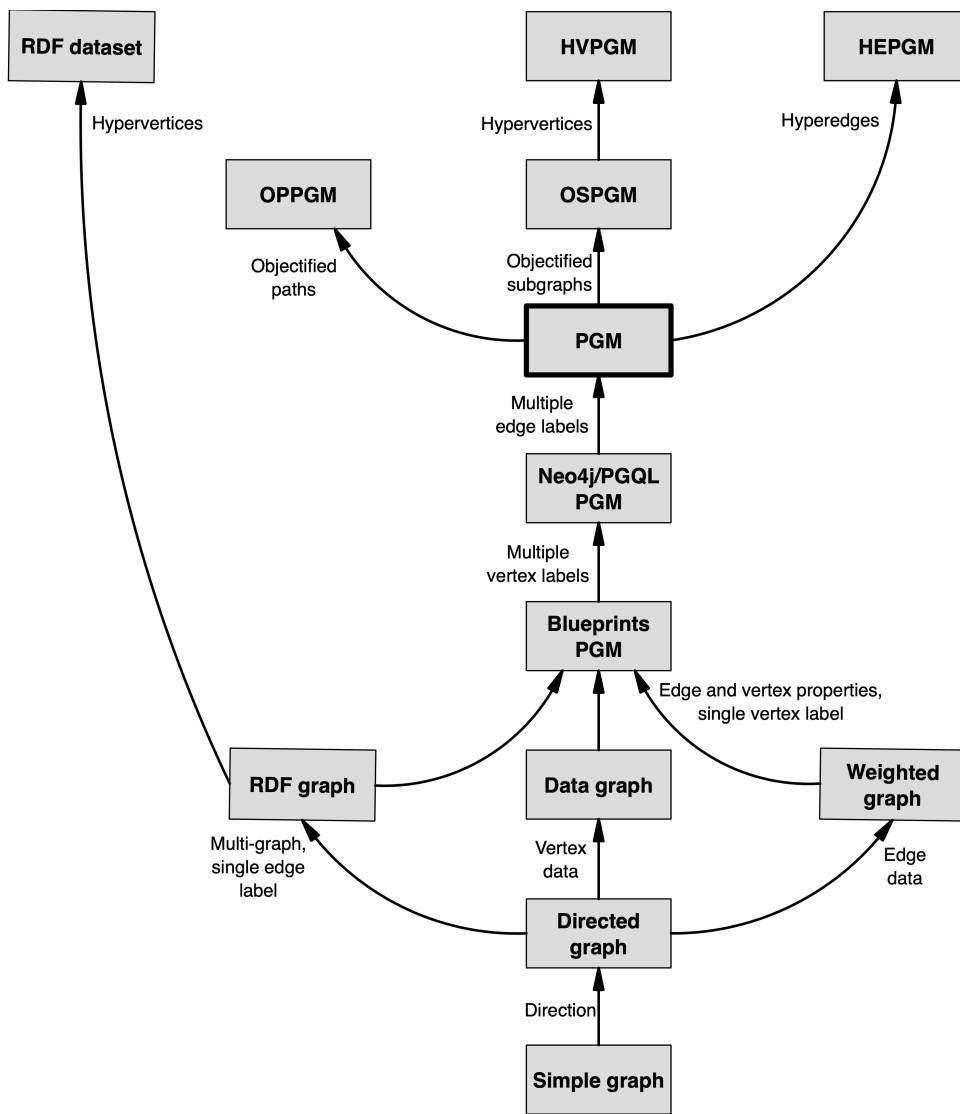












Property Graph Model (PGM)

- Good balance between
 - Expressiveness
 - Complexity
- High data modelling ability
- Still simple and intuitive

Conclusion

- Many graph models have been defined
- Expressiveness and complexity are competing yard-sticks
- The PGM strikes a good balance
- This encourages future extensions of practical systems, such as Neo4j



Some References

Recent Survey

Claudio Gutierrez, Jan Hidders, and Peter Wood. Graph data models. In *Encyclopedia of Big Data Technologies*, Springer, 2018. DOI: https://doi.org/10.1007/978-3-319-63962-8_81-1.

Industry Adopters of PGM

- Neo4j: <http://neo4j.com/>
- Apache TinkerPop: <http://tinkerpop.apache.org/>
- IBM Graph: <https://ibm-graph-docs.ng.bluemix.net/>
- Oralde: Oskar van Rest, Sungpack Hong, Jinha Kim, Xuming Meng, and Hassan Chafi. PGQL: A property graph query language. In GRADES, page 7, ACM, 2016.
<https://doi.org/10.1145/2960414.2960421>
- SAP HANA: Michael Rudolf, Marcus Paradies, Christof Bornhövd, and Wolfgang Lehner. The graph story of the SAP HANA database. In BTW, vol. 214, pages 403420, GI, 2013.
- Sparksee: <http://www.sparsity-technologies.com/>